Aura Protocol: Hybrid Post-Quantum Secure Messaging with Ratchet-Level KEM Key Renewal

Oleksandr Melnychenko*

Khmelnytskyi National University, Khmelnytskyi, Ukraine

Abstract

Long-lived end-to-end encrypted conversations remain exposed to harvest-now-decrypt-later attacks when post-quantum protection is confined to session establishment. Aura Protocol uses an authenticated X25519/ML-KEM-768 pre-key exchange and renews both sources at each directed transition rather than on a sparse or periodic schedule. It retains classical Ed25519 identity authentication, so post-quantum claims concern session-secret confidentiality, not peer authentication. Each transition binds the advertised KEM public key through HKDF and derives separate payload and metadata keys. After local compromise, the first honest directed transition restores a classical contribution. Hybrid recovery requires a later transition using peer DH and KEM material generated after the compromise. Six reduction-style arguments rely on Gap-CDH, ML-KEM IND-CCA2, explicit two-source and split-input PRF assumptions for HKDF, and misuse-resistant authenticated-encryption assumptions. Reduced Tamarin and ProVerif models cover selected secrecy and correspondence properties. Implementation tests address parsing, persistence, and replay behavior. On the evaluation host, the handshake takes 1.1 ms, a hybrid transition takes 259 μs, and alternating 256-byte messages take 280 μs per message. A hybrid ratchet carries 2,304 bytes of raw DH/KEM material before serialization framing.

Keywords: post-quantum cryptography, secure messaging, Double Ratchet, ML-KEM, post-compromise security, formal verification

*Corresponding author.

Email address: melnychenko@khmnu.edu.ua (Oleksandr Melnychenko)

1. Introduction

Modern asynchronous end-to-end encrypted messaging commonly combines authenticated pre-key agreement with a stateful ratchet to obtain forward secrecy and post-compromise security under elliptic-curve Diffie–Hellman assumptions. X3DH and the Double Ratchet are the standard reference designs for that pattern [1, 2]. This design discipline is now used beyond ordinary interpersonal chat: mobile endpoints, operational monitoring systems, and sensor-assisted field workflows increasingly rely on asynchronous encrypted channels for coordination data, alerts, and inspection-derived evidence [3, 4, 5]. In these settings, transcripts may remain valuable long after delivery, while endpoint images and backups may persist for years.

A cryptographically relevant quantum computer (CRQC) would undermine the classical assumption behind ECDH-based session secrecy. NIST has standardized ML-KEM as a lattice-based key-encapsulation mechanism [6], but inserting a KEM into a stateful messaging protocol is not a syntactic replacement for DH. A KEM introduces different directionality, state exposure, ciphertext, and public-key-binding requirements. Aura Protocol is designed around four resulting protocol constraints:

Challenge 1: Harvest-now-decrypt-later attacks. An adversary that records ECDH-based traffic today could later recover affected session secrets if a CRQC becomes available. Classical forward secrecy removes old DH private values from endpoint state, but it does not protect a recorded ECDH transcript once the elliptic-curve discrete logarithm problem becomes tractable through Shor's algorithm [7].

Challenge 2: KEM integration granularity. Adding a post-quantum KEM only to the initial handshake protects session establishment, but it leaves later post-compromise recovery to the classical DH ratchet. PQXDH [8] is therefore a handshake baseline, whereas sparse post-quantum ratchets such as SPQR/Triple Ratchet [9, 10, 11] and periodic in-band rekeying designs such as Apple PQ3 [12, 13, 14] are closer comparisons for continuing post-quantum state evolution. Aura Protocol belongs to this second class: post-quantum material is refreshed inside the ratchet state, not only at session creation.

Challenge 3: Directed KEM recovery in this construction. Unlike DH, a KEM requires the encapsulation public key to be available before the peer can contribute a ciphertext. Aura Protocol therefore uses a directed KEM-public-key

schedule and states the recovery boundary explicitly. After a local endpoint compromise, the next honest directed step restores the classical DH contribution. After full two-endpoint state compromise, the first post-compromise directed step may still consume peer DH and KEM state exposed by the compromise. Hybrid recovery is therefore tied to a later directed step using peer material generated after the compromise. This distinction determines the PCS claim in security claim 6.7.

Challenge 4: Metadata leakage. Standard Double Ratchet envelopes expose ratchet public keys and message counters in cleartext headers [10]. Payload encryption alone does not hide ordering, message lifecycle, or application-level correlation fields. Aura Protocol therefore adds an encrypted inner metadata layer for message index, payload nonce, message type, and correlation identifiers, while leaving delivery metadata, sizes, timing, and ratchet public material visible.

Objective. This paper specifies and evaluates a two-party asynchronous messaging protocol that refreshes post-quantum key material at ratchet transitions and distinguishes the recovery conditions following local and two-endpoint compromise. The evaluation combines reduction-style analysis, symbolic models, executable validation, and measurements of the implemented state machine.

Scientific novelty. Relative to sparse post-quantum ratcheting (SPQR/ML-KEM Braid [9, 10, 11]) and periodic in-band rekeying (Apple PQ3 [12, 13, 14]), the central scientific novelty is a directed hybrid-ratchet state-transition method. It couples each direction change to joint X25519/ML-KEM renewal, binds the newly advertised KEM public key into the derived state, and gives an explicit two-case PCS statement: one-step classical recovery after local compromise and delayed hybrid recovery after two-endpoint compromise. The novelty is construction-level rather than a claim of a new cryptographic primitive. It is developed through three method-level contributions:

1. **N1: directed hybrid transition and PCS characterization.** Every direction change advances the root state with X25519 and ML-KEM-768 contributions. The resulting PCS characterization separates one-step classical recovery after local compromise from delayed hybrid recovery after full two-endpoint compromise (security claim 6.7).
2. **N2: ratchet KEM-key state binding.** Each newly advertised ML-KEM public key enters the HKDF info parameter during ratchet key

derivation. Substitution changes the derived root state and causes the subsequent AEAD check to fail, which binds the accepted KEM key without a per-ratchet signature (Lemma 4.1).

3. **N3: ratchet-scoped metadata-key separation.** Envelope metadata and payload bytes use separate key domains. The metadata key rotates by ratchet epoch, and both layers use AES-256-GCM-SIV to limit the consequences of nonce reuse (Lemma 4.2).

Validation contribution (V1). The three method-level contributions are evaluated through reduction-style arguments, reduced symbolic models, executable checks of parsing, ratchet atomicity, replay handling and sealed-state persistence, and measurements of the two-party implementation.

Table 1: Claim-to-evidence map for the two-party Aura Protocol scope.

ID	Research claim	Main evidence in this paper
N1	A direction-triggered X25519/ML-KEM state transition yields distinct local and two-endpoint PCS recovery conditions.	PCS claim (security claim 6.7), ratchet algorithm (Algorithm 2), ratchet benchmarks (Tables 12 and 14).
N2	HKDF-info binding makes KEM public-key substitution a state mismatch rather than a silent downgrade.	Augmented-binding lemma (Lemma 4.1), manipulation traces (Table 3), tamper/rejection validation.
N3	Ratchet-scoped metadata keys separate envelope metadata from per-message payload protection.	Two-layer AEAD lemma (Lemma 4.2), metadata taxonomy (Table 4), envelope processing tests.
V1	The implemented state machine preserves the protocol's rejection, replay, persistence, and cost assumptions.	Verification-scope table (Table 6), validation map (Table 16), operational invariants (Section 10), and benchmark results.

2. Related Work

Signal protocol. The Signal protocol, formalized by Cohn-Gordon et al. [2], achieves forward secrecy and post-compromise security through the combination of X3DH key agreement [1] and the Double Ratchet algorithm. Its

ratcheting behavior has received detailed symbolic and computational analysis [2, 15, 16]. Its DH-derived session keys do not protect recorded traffic against a future adversary able to solve elliptic-curve discrete logarithms.

Post-quantum extensions to Signal. Signal's PQXDH protocol [8] adds a post-quantum KEM encapsulation at session establishment, with Kyber-1024 used in the specification's example instantiation. Brendel et al. [17] propose a generic post-quantum X3DH construction, and Hashimoto et al. [18] provide an efficient instantiation. These works are natural baselines for harvest-now-decrypt-later protection at session establishment, but not for post-quantum post-compromise recovery in a long-lived channel.

Signal SPQR/Triple Ratchet [9, 10] is the relevant Signal-family comparison point for ongoing post-quantum state evolution. It runs a sparse post-quantum ratchet alongside the classical Double Ratchet and combines their message keys. Its ML-KEM Braid component is specified as a Sparse Continuous Key Agreement protocol [11]. For a current comparison, PQXDH is therefore only the handshake baseline.

Industrial post-quantum messaging. Apple PQ3 [12, 13, 14] also moves post-quantum material beyond the initial handshake through periodic in-band rekeying, and has public reduction-style and Tamarin analyses. It is a deployed baseline with a closed implementation. Aura Protocol occupies a complementary position: it gives an open two-party construction whose distinctive mechanisms are ratchet-level KEM-key binding, metadata-key rotation, and a separately stated validation boundary.

Ratcheting analysis. Alwen et al. [15] and Bienstock et al. [16] give detailed analyses of Double Ratchet security notions, modularization, and post-compromise recovery. This line of work motivates the separation between the state-evolution claim and the concrete implementation invariants studied here.

Hybrid key exchange. The concept of hybrid key exchange—combining classical and post-quantum primitives—has been studied for Internet key exchange and TLS, including early CECPQ2 deployment experiments [19], Open Quantum Safe work [20], hybrid KEM/AKE analyses [21], and formal TLS 1.3 analyses [22]. These works establish security under the assumption that at least one component remains secure. The combiner in security claim 6.1 follows the same principle, using HKDF with the classical shared secret as salt and the KEM shared secret as input keying material.

Formal verification of messaging protocols. Tamarin [23] and ProVerif [24] have been used to verify properties of the Signal protocol [2]. The models in this paper adapt that line of analysis to a hybrid post-quantum setting, including custom equations for ML-KEM-768 and the Tamarin source-saturation issues caused by multiple DH operations.

Metadata privacy and anonymous delivery. Sealed Sender [25] hides sender identity from Signal's service, while mix-network designs such as Sphinx [26] and Loopix [27] address network-level anonymity. Aura Protocol's metadata layer is narrower: it encrypts protocol envelope fields under rotating per-ratchet-epoch keys, complementing rather than replacing sender-hiding delivery systems.

Nonce-misuse-resistant AEAD. AES-256-GCM-SIV [28] provides misuse-resistant authenticated encryption, degrading to deterministic authenticated encryption under nonce reuse, whereas AES-GCM loses its standard confidentiality and integrity bounds under repeated nonces. Rogaway and Shrimpton [29] formalize the MRAE security notion. Aura Protocol uses GCM-SIV to provide residual authenticity and deterministic leakage bounds alongside monotonic nonce counters.

3. Preliminaries

3.1. Notation

Let λ denote the security parameter. A function $f(\lambda)$ is *negligible* if $f(\lambda) \in o(\lambda^{-c})$ for every constant $c > 0$. We write this as $f \in \mathrm{negl}(\lambda)$. All algorithms are implicitly parameterized by λ. The notation $x \xleftarrow{\$} S$ denotes uniform random sampling from a finite set S. The operator $\|$ denotes byte-string concatenation, and PPT denotes probabilistic polynomial-time.

An *epoch* in this paper means a ratchet epoch: the interval of session state governed by one root key, one active sending or receiving chain key, and one metadata key. The epoch counter increments when a directed hybrid ratchet step installs fresh DH and ML-KEM material. Multiple same-direction messages may belong to the same epoch. They are distinguished by the per-chain message index n.

Table 2: Protocol-specific notation.

Symbol	Meaning
IK_X	Long-term X25519 identity keypair of party X
SPK_X	Signed pre-key (X25519) of party X
EK_X	Ephemeral X25519 keypair of party X
OPK_X	One-time pre-key (X25519) of party X
(kpk_X, ksk_X)	ML-KEM-768 keypair of party X
e	Ratchet epoch counter
rk_e	Root key at epoch e
ck_e	Chain key at epoch e
$mk_{e,n}$	Message key at epoch e, chain index n
mdk_e	Metadata key at epoch e
$R(\cdot)$	Hybrid ratchet step function
q_s	Number of sessions
q_m	Total encrypted message attempts across all sessions
$N_{\max}$	Negotiated per-chain message limit

3.2. Cryptographic Primitives

Definition 3.1 (Diffie–Hellman on X25519)**.** The proof uses an idealized prime-order X25519 abstraction with

$$q = 2^{252} + 27742317777372353535851937790883648493.$$

Curve25519 has cofactor 8. The construction follows RFC 7748 clamping and rejects malformed field encodings, small-order inputs, and all-zero shared secrets where applicable. Cofactor and twist subtleties are not modeled beyond this abstraction. The X25519 function [30] computes scalar multiplication $\mathrm{X25519}(a, B) = [a]B$ with base point $u = 9$.

Definition 3.2 (Gap-CDH)**.** The Gap-CDH assumption states that for all PPT adversaries $\mathcal{A}$ with access to a DDH oracle $\mathcal{O}_{\mathrm{DDH}}$:

$$\begin{aligned}\mathbf{Adv}^{\text{Gap-CDH}}_{\text{X25519}}(\mathcal{A}) &= \Pr\left[\mathcal{A}^{\mathcal{O}_{\mathrm{DDH}}}(G, [a]G, [b]G) = [ab]G\right] \\ &\leq \mathrm{negl}(\lambda), \qquad a, b \xleftarrow{\$} \mathbb{Z}_q. \end{aligned} \tag{1}$$

Definition 3.3 (Key Encapsulation Mechanism)**.** A key encapsulation mechanism Π is specified by three algorithms with the following interfaces:

$$\begin{aligned}\mathrm{KeyGen}() &\to (pk, sk), \\ \mathrm{Encaps}(pk) &\to (ct, ss), \\ \mathrm{Decaps}(sk, ct) &\to ss.\end{aligned}$$

Correctness requires

$$(ct, ss) \leftarrow \mathrm{Encaps}(pk),$$
$$\mathrm{Decaps}(sk, ct) = ss.$$

Definition 3.4 (IND-CCA2 for KEM)**.** For a key encapsulation mechanism Π, the IND-CCA2 advantage of an adversary $\mathcal{A}$ is defined as

$$\mathbf{Adv}_{\Pi}^{\text{IND-CCA2}}(\mathcal{A}) = \left| \Pr\left[\mathcal{A}^{\mathcal{O}_{\text{Decaps}}}(pk, ct^*, ss_b) = b\right] - \frac{1}{2} \right| \leq \mathrm{negl}(\lambda), \quad (2)$$

where $b \xleftarrow{\$} \{0,1\}$, $(ct^*, ss_0) \leftarrow \mathrm{Encaps}(pk)$, $ss_1 \xleftarrow{\$} \{0,1\}^{256}$, and $\mathcal{A}$ may not query $\mathcal{O}_{\text{Decaps}}$ on ct^*.

Definition 3.5 (Two-Source HKDF Assumption)**.** For the argument order used in this construction, define

$$F(s, k, \mathit{info}) = \mathrm{HKDF}(k, 32, s, \mathit{info}),$$

where s is passed as the RFC 5869 salt and k as input keying material. We assume that F has the two-source PRF property: its output is computationally indistinguishable from random when either s or k is uniform and independent, even if the other source is known or selected without access to the fresh secret. RFC 5869 defines the extract-and-expand interface [32], and the HKDF analysis supplies its constructional background [31]. Neither reference is cited here as a theorem for this exact two-source combiner. The property is an explicit assumption of the security arguments below.

Definition 3.6 (Split-Input Ratchet-KDF Assumption)**.** For the ratchet argument order, define

$$G_L(s, x, y, \mathit{info}) = \mathrm{HKDF}(x \,\|\, y, L, s, \mathit{info}),$$

where s is the previous root key, x is the X25519 contribution, and y is the ML-KEM contribution. We assume that G_L has the split-input PRF property: its output is computationally indistinguishable from random when either x or y is uniform and independent, even if s and the other contribution are known or selected without access to the fresh secret. This component-robust property is not implied by the two-source PRF property in Definition 3.5: neither the salt nor the complete concatenated input need be uniform in the ratchet game. It is an explicit assumption of the ratchet PCS arguments rather than a theorem attributed to RFC 5869.

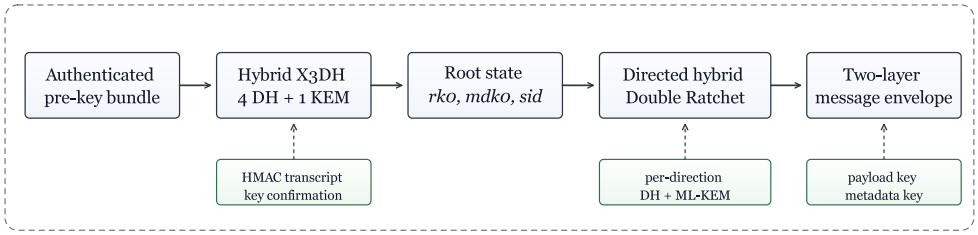

Figure 1: Protocol phases and cryptographic inputs used by Aura Protocol.

Definition 3.7 (MRAE Security [28, 29])**.** AES-256-GCM-SIV satisfies MRAE security if:

- Under unique nonces: full IND-CPA + INT-CTXT (standard AEAD security).
- Under nonce reuse: degrades to deterministic authenticated encryption (DAE), leaking equality patterns for repeated $(nonce, AAD, plaintext)$ tuples while authenticity remains governed by the MRAE bound.

4. Protocol Design

Aura Protocol consists of two phases: a *hybrid pre-key handshake* (Section 4.2) that establishes initial shared state, and a *hybrid Double Ratchet* (Section 4.3) that continuously refreshes key material. Messages are protected by a *two-layer AEAD* construction (Section 4.4). Figure 1 summarizes the protocol phases and the key material refreshed in each phase.

4.1. Pre-Key Bundle

Each party R publishes a pre-key bundle:

$$\begin{aligned} Bundle_R = \Big(&v,\ ed_R^{pub},\ IK_R^{pub},\ spk\_id,\ SPK_R^{pub},\ \sigma_{SPK},\\ &\{(opk\_id_j, OPK_{R,j}^{pub})\}_j,\ kpk_R,\ \sigma_{IKX}\Big). \end{aligned} \tag{3}$$

where $\sigma_{IKX} = \text{Ed25519.Sign}(ed_R^{sk}, IK_R^{pub})$ binds the X25519 identity key to the Ed25519 identity key, $\sigma_{SPK} = \text{Ed25519.Sign}(ed_R^{sk}, SPK_R^{pub})$ signs the medium-term pre-key, $\{OPK_{R,j}\}$ is a set of one-time pre-keys, and kpk_R is the ML-KEM public key. The KEM public key is validated and transcript-bound by

key confirmation. It is not treated as a separately Ed25519-signed object in this construction.

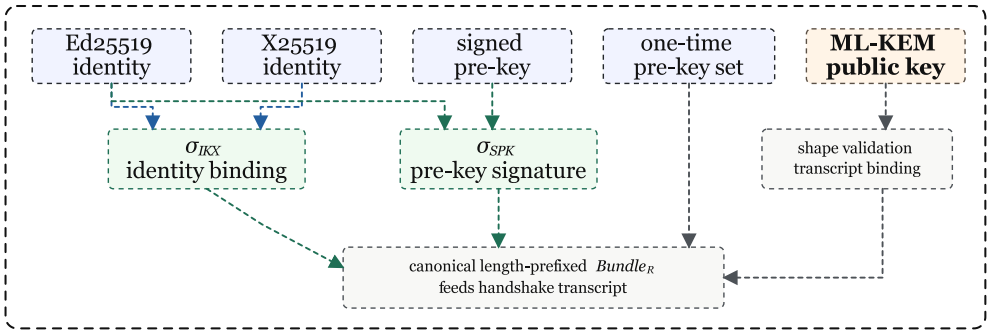

Figure 2: Pre-key bundle anatomy. Classical identity and signed-pre-key fields are directly signed, while the ML-KEM public key is validated as protocol input and bound by the handshake transcript and key-confirmation MAC.

4.2. Hybrid Pre-Key Agreement

The initial key agreement follows the authenticated asynchronous pre-key pattern used by X3DH [1] and combines four X25519 Diffie–Hellman contributions with an ML-KEM-768 encapsulation and bilateral key confirmation. The message flow and root-key inputs are shown in Figure 3.

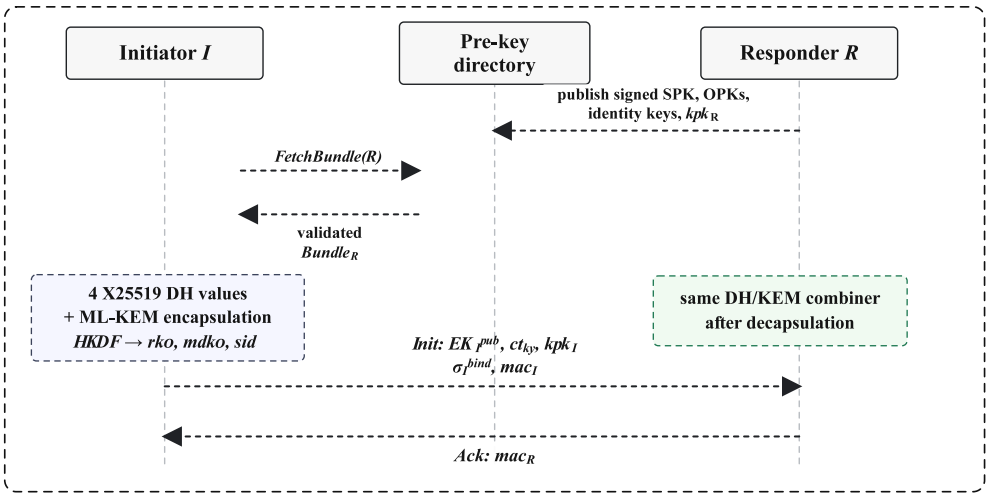

Figure 3: Hybrid pre-key handshake messages and root-key inputs. The KEM public key is validated and transcript-bound by key confirmation. It is not a separate per-session signature.

Algorithm 1: Initiator computation for the hybrid pre-key agreement.

Inputs. Initiator identity (ed_I, IK_I), local KEM key kpk_I, and responder bundle $Bundle_R$.

Outputs. Initial state (rk_0, mdk_0, sid) and $Init$ message.

1. Validate responder material. Accept $Bundle_R$ only if the Ed25519 bindings verify, all X25519 public keys pass small-order rejection, and kpk_R has a valid ML-KEM public-key encoding.

2. Compute the classical pre-key contribution. Generate (ek_I, EK_I^{pub}) and select OPK_R if present. Then compute

$$\begin{aligned} dh_1 &= \mathrm{X25519}(ik_I, SPK_R^{pub}), \\ dh_2 &= \mathrm{X25519}(ek_I, IK_R^{pub}), \\ dh_3 &= \mathrm{X25519}(ek_I, SPK_R^{pub}), \\ dh_4 &= \mathrm{X25519}(ek_I, OPK_R^{pub}) \quad \text{if available}, \\ IKM &= \mathrm{0xFF}^{32} \,\|\, dh_1 \,\|\, dh_2 \,\|\, dh_3[\,\|\, dh_4], \\ cs &= \mathrm{HKDF}(IKM, 32, \varepsilon, \text{“Aura-X3DH”}). \end{aligned}$$

3. Add the post-quantum contribution and derive session state.

$$\begin{aligned} (ct_{ky}, ss_{ky}) &\leftarrow \text{ML-KEM-768.Encaps}(kpk_R), \\ rk_0 &\leftarrow \mathrm{HKDF}(ss_{ky}, 32, cs, \text{“Aura-Hybrid-X3DH”}), \\ sid &\leftarrow \mathrm{HKDF}(rk_0, 16, ct_{ky}, \text{“Aura-SessionId”}), \\ ctx_{md} &\leftarrow \mathrm{sort}(EK_I^{pub}, SPK_R^{pub}) \,\|\, sid, \\ mdk_0 &\leftarrow \mathrm{HKDF}(rk_0, 32, ctx_{md}, \text{“Aura-MetadataKey”}), \\ kc_I &\leftarrow \mathrm{HKDF}(rk_0, 32, \varepsilon, \text{“Aura-KeyConfirm-I”}), \\ kc_R &\leftarrow \mathrm{HKDF}(rk_0, 32, \varepsilon, \text{“Aura-KeyConfirm-R”}). \end{aligned}$$

4. Bind the initiator identity and transcript.

$$\begin{aligned} m_I^{bind} &\leftarrow \text{“Aura-Handshake-Init-Identity-Binding”} \,\|\, ed_I^{pub} \,\|\, IK_I^{pub} \,\|\, kpk_I, \\ \sigma_I^{bind} &\leftarrow \text{Ed25519.Sign}(ed_I^{sk}, m_I^{bind}), \\ Init_0 &\leftarrow (ed_I^{pub}, IK_I^{pub}, EK_I^{pub}, ct_{ky}, kpk_I, \\ &\qquad \sigma_I^{bind}, opk\_id, N_{\max}), \\ th &\leftarrow \mathrm{HMAC}(\text{“Aura-Handshake-Transcript”}, \mathrm{LP}(Bundle_R) \,\|\, \mathrm{LP}(Init_0)), \\ mac_I &\leftarrow \mathrm{HMAC}(kc_I, th), \\ mac_R^* &\leftarrow \mathrm{HMAC}(kc_R, th), \\ Init &\leftarrow (Init_0, mac_I). \end{aligned}$$

Here LP(x) denotes the length-prefixed canonical encoding used by the transcript serializer. Throughout the pseudocode, HKDF($ikm, L, salt, info$) means RFC 5869 Extract with $(salt, ikm)$ followed by Expand to L bytes with the stated *info* string. The initiator retains mac_R^* for the responder acknowledgement and keeps ss_{ky} only in pending state for the initial post-handshake ratchet. Ephemeral DH state and intermediate shared secrets are erased after the derivation completes.

The responder performs the symmetric computation: it recomputes the four DH shared secrets using its own private keys, decapsulates the ML-KEM ciphertext, derives the same root key rk_0, verifies mac_I, and sends $Ack = (mac_R)$. The initiator finalizes by verifying mac_R in constant time.

Hybrid combiner design. The combiner uses the classical DH output cs as the HKDF salt and the ML-KEM shared secret ss_{ky} as input keying material. This ordering follows the RFC 5869 interface [32]. Its robustness argument invokes the two-source PRF assumption in Definition 3.5, not the ordinary extractor reading in which the salt is public and independent. If the classical reduction path applies, cs supplies the fresh source. If the ML-KEM reduction path applies, ss_{ky} supplies it. The construction therefore has two alternative security arguments, each conditioned on the exact HKDF assumption stated above.

Key confirmation. Bilateral key confirmation prevents unknown key-share (UKS) attacks [33]. The transcript digest th is computed with HMAC over length-prefixed inputs. Authentication is provided by the later HMAC under keys kc_I and kc_R. These confirmation keys are derived from rk_0 using distinct HKDF info strings, ensuring domain separation.

DH validation. All received X25519 public keys are validated by checking against the eight small-order points of Curve25519 (points of order dividing the cofactor 8) using constant-time comparison to avoid data-dependent timing behavior. Additionally, points are verified to satisfy $key < p$ where $p = 2^{255} - 19$, using branchless word-by-word subtraction.

4.3. Hybrid Double Ratchet

After the handshake, communication proceeds via the hybrid Double Ratchet, which combines a symmetric-key chain ratchet with an asymmetric hybrid ratchet.

4.3.1. Symmetric Chain Ratchet

Message keys are derived from the chain key via HKDF with distinct info strings for domain separation:

$$mk_{e,n} = \text{HKDF}(ck_e^{(n)},\ 32,\ \varepsilon,\ \text{“Aura-Msg”}), \tag{4}$$

$$ck_e^{(n+1)} = \text{HKDF}(ck_e^{(n)},\ 32,\ \varepsilon,\ \text{“Aura-Chain”}). \tag{5}$$

The old chain key $ck_e^{(n)}$ is securely erased after derivation, providing per-message forward secrecy within an epoch.

4.3.2. Asymmetric Hybrid Ratchet

Algorithm 2: Send-side hybrid ratchet transition (epoch $e \to e{+}1$).

Inputs. Current root key rk_e and peer ratchet public keys (D_{peer}, kpk_{peer}).

Outputs. Updated sender state $(rk_{e+1}, ck_{e+1}, mdk_{e+1}^{send})$ and ratchet header Hdr_{e+1}.

1. Generate fresh advertised sender material.

$$(d_{e+1}, D_{e+1}) \leftarrow \text{X25519.KeyGen}(),$$
$$(kpk_{e+1}, ksk_{e+1}) \leftarrow \text{ML-KEM-768.KeyGen}().$$

2. Compute directed hybrid input material.

$$dh_{e+1} \leftarrow \text{X25519}(d_{e+1}, D_{peer}),$$
$$(ct_{e+1}, ss_{e+1}) \leftarrow \text{ML-KEM-768.Encaps}(kpk_{peer}),$$
$$ikm_{e+1} \leftarrow dh_{e+1} \,\|\, ss_{e+1}.$$

3. Bind the next KEM public key into the KDF transcript.

$$info_{e+1} \leftarrow \text{“Aura-Hybrid-Ratchet”} \,\|\, kpk_{e+1},$$
$$T_{e+1} \leftarrow \text{HKDF}(ikm_{e+1}, 96, rk_e, info_{e+1}),$$
$$(rk_{e+1}, ck_{e+1}, mdk_{e+1}^{send}) \leftarrow \text{split}_{32,32,32}(T_{e+1}).$$

4. Advertise the ratchet header.

$$Hdr_{e+1} \leftarrow (D_{e+1}, ct_{e+1}, kpk_{e+1}).$$

The sender attaches Hdr_{e+1} to the next envelope, installs the new epoch state, and erases the previous DH/KEM private state, rk_e, dh_{e+1}, and ss_{e+1} after derivation. The recovery arguments for this transition use the split-input PRF assumption in Definition 3.6, since only one component of $dh_{e+1} \,\|\, ss_{e+1}$ may remain secret.

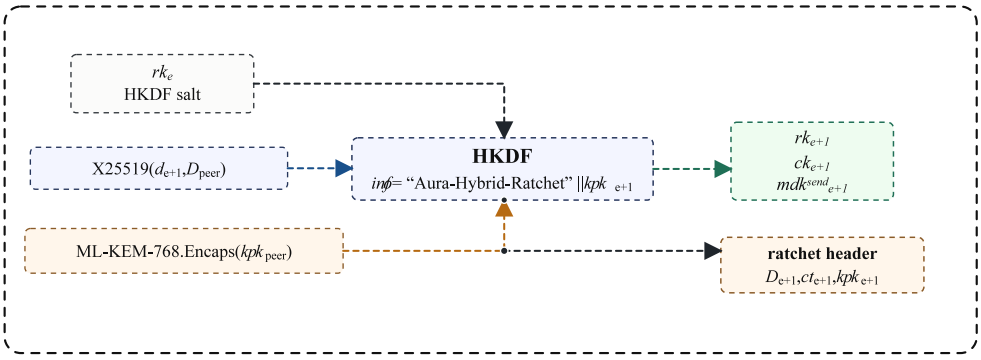

Figure 4: Send-side hybrid ratchet derivation and transmitted ratchet header. The new KEM public key is part of the HKDF info string, binding the advertised post-quantum state to the derived root.

Ratchet trigger. Classical DH ratchets normally advance the asymmetric step when a new peer DH public key is received [10]. Aura Protocol uses a single trigger rule for this directed hybrid design: after every received message, the next outbound message executes a full X25519+ML-KEM ratchet and advertises the resulting ratchet header. A chain-exhaustion ratchet is also triggered when the negotiated per-chain limit $N_{\max}$ is reached.

Augmented info string. The HKDF info parameter includes the new ML-KEM public key kpk_{e+1} in Algorithm 2 and Figure 4. This binding supports the following lemma:

Lemma 4.1 (Augmented HKDF Info Binding)**.** *If an adversary substitutes $kpk' \neq kpk_{e+1}$ in the ratchet header, the receiver's derived root key is computationally independent of the sender's root key, except with negligible HKDF-output collision probability. Consequently, all derived chain and message keys differ, and AEAD decryption fails under the* INT-CTXT *property of* AES-256-GCM-SIV*. This provides an implicit state-binding check for the KEM public key without per-step signatures. It does not provide a separate identity-authentication mechanism.*

Table 3: Operational traces for KEM public-key manipulation.

Adversarial action	Detection point	Security effect
Substitute the responder's handshake KEM public key in the directory while keeping signed X25519 material unchanged.	The peers derive different handshake roots, so bilateral key-confirmation MACs do not validate.	The session is not established. This is denial of service, not a silent confidentiality downgrade.
Replace kpk_{e+1} in a ratchet header.	The receiver uses kpk'_{e+1} in HKDF info and derives a different root, chain key, and metadata key. Metadata or payload AEAD fails.	The candidate transition is rejected and the previous state is restored.
Replay an old ratchet header from the same session.	Epoch, nonce, replay-window, and cached-key checks are evaluated before candidate state is retained.	Admissible delayed traffic uses bounded compatibility state. Stale state-changing headers are rejected.
Omit or truncate KEM material to force a classical-only path.	Version, length, and profile checks fail before KDF input is accepted.	Hybrid policy enforcement prevents transparent fallback to a weaker transcript.

Receive-side ratchet. The receiver evaluates a ratchet header against a temporary in-memory snapshot. The candidate transition is retained only after syntactic validation, KEM decapsulation, DH derivation, metadata and payload decryption, and replay checks succeed. Durable export is a separate caller-controlled operation and is not part of the decrypt call's atomic boundary. The executable evidence for the pinned snapshot covers adversarial validation and AEAD failures after candidate derivation; it does not claim atomic recovery from arbitrary allocator, platform, or process failures.

Algorithm 3: Receive-side hybrid ratchet and transactional acceptance.

Inputs. Current state S_e, envelope C, optional ratchet header $Hdr_{e+1} = (D_{e+1}, ct_{e+1}, kpk_{e+1})$.
Outputs. Plaintext M and committed state S'. On rejection, return $\perp$ and the unchanged state S_e.
$S_{old} \leftarrow S_e$;
if *C has malformed version, length, epoch, or public-key fields* **then**
  return $(\perp, S_e)$;
if *Hdr_{e+1} is present* **then**
  $ss_{e+1} \leftarrow$ ML-KEM-768.Decaps$(ct_{e+1}, ksk_e^{local})$;
  $dh_{e+1} \leftarrow$ X25519(d_e^{local}, D_{e+1});
  $info_{e+1} \leftarrow$ "Aura-Hybrid-Ratchet" $\| \, kpk_{e+1}$;
  $T_{e+1} \leftarrow$ HKDF$(dh_{e+1} \| \, ss_{e+1}, 96, rk_e, info_{e+1})$;
  Install candidate $(rk_{e+1}, ck_{e+1}, mdk_{e+1}^{recv})$ and cache old metadata keys for bounded delayed delivery;
Derive or recover the candidate message key $mk_{e,n}$ for the envelope epoch and index;
if *the nonce was already accepted or the replay window cannot represent it* **then**
  restore S_{old} and **return** $(\perp, S_e)$;
if *metadata or payload AEAD decryption fails* **then**
  restore S_{old} and **return** $(\perp, S_e)$;
Commit replay state, skipped-key state, and root/chain/metadata keys in memory;
Erase consumed DH, KEM, HKDF, and message-key material;
return (M, S');

Transactional rollback. Before applying a received ratchet, the receiver records a temporary copy of the current session state: root key, chain keys, DH/ML-KEM private keys, skipped message keys, and metadata keys. If metadata or payload decryption fails after the candidate ratchet is applied, the previous state is restored. A forged ratchet envelope therefore cannot desynchronize the in-memory session. Crash consistency and rollback freshness additionally require successful sealed-state export and an external monotonic counter as described in Section 4.6.

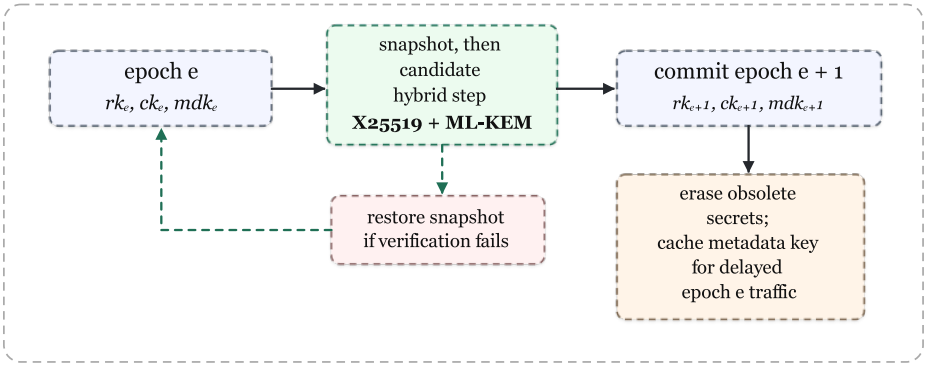

Figure 5: Ratchet state lifecycle. A successful hybrid step commits new root, chain, and metadata keys. Obsolete high-value secrets are erased after authentication, while bounded compatibility state is retained for delayed traffic and rollback safety.

4.3.3. Session Initialization

After the hybrid pre-key handshake produces rk_0 and the ML-KEM shared secret, the session is initialized via an initial hybrid ratchet step:

$$dh_{init} = \text{X25519}(d_{local}, D_{remote}), \tag{6}$$

$$hybrid\_ikm = dh_{init} \parallel ss_{ky}, \tag{7}$$

$$T_{init} = \text{HKDF}(hybrid\_ikm,\ 64,\ rk_0,\ \text{“Aura-Hybrid-Ratchet”}), \tag{8}$$

$$rk_1 = T_{init}[0{:}32], \tag{9}$$

$$(ck_{send},\ ck_{recv}) = \text{HKDF}(rk_1,\ 64,\ \varepsilon,\ \text{“Aura-ChainInit”}). \tag{10}$$

The final post-handshake root used by the message layer is rk_1. The value mdk_0 remains the initial metadata key until the first ratchet. The initiator uses the first 32 bytes of the “Aura-ChainInit” output as ck_{send} and the last 32 bytes as ck_{recv}. The responder swaps them. The AKE claim establishes pseudorandomness of rk_0. Post-handshake initialization carries this property to $(rk_1, ck_{send}, ck_{recv})$ through HKDF domain separation with rk_0 as salt.

4.4. Two-Layer AEAD Encryption

Each message envelope consists of two independently encrypted layers:

Layer 1: Metadata.

$$ct_{md} = \text{AES-256-GCM-SIV.Enc}(mdk_e^{send},\ nonce_{md},\ metadata,\ aad_{md}), \tag{11}$$

where $aad_{md} = sid \,\|\, ibh \,\|\, epoch \,\|\, v$ includes the session ID, identity binding hash, epoch counter, and protocol version. The metadata contains the message index, payload nonce, envelope type, and correlation identifiers.

Layer 2: Payload.

$$ct_{pay} = \text{AES-256-GCM-SIV.Enc}(mk_{e,n},\ nonce_{pay},\ plaintext,\ aad_{pay}), \quad (12)$$

where $aad_{pay} = sid \,\|\, ibh \,\|\, epoch \,\|\, n \,\|\, v$ additionally includes the message index n.

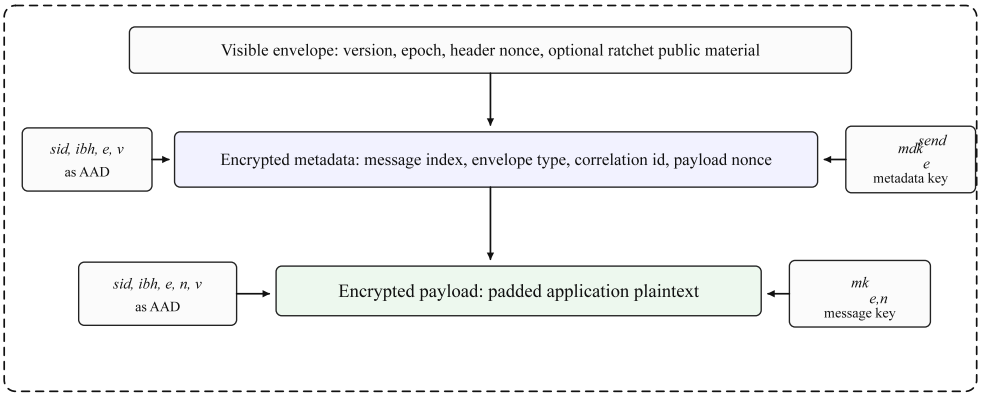

Figure 6: Envelope protection split: visible transport fields, encrypted metadata, and encrypted payload. Delivery timing, sizes, and ratchet public material remain visible. Sender metadata and application content are protected by separate AEAD keys.

Table 4: Metadata-confidentiality boundary of the two-layer envelope.

Class	Examples	Protection statement
Protected by the metadata AEAD	Message index, payload nonce, envelope type, application correlation identifiers.	Confidential and integrity-protected under the per-epoch metadata key mdk_e, with session, identity, epoch, and version bound as AAD.
Protected by the payload AEAD	Application plaintext and application-level confidential content.	Confidential and integrity-protected under per-message keys $mk_{e,n}$ derived from the chain key.
Partially observable through traffic shape	Direction changes, burst structure, retries, and approximate conversation activity.	Not directly encoded in plaintext fields, but may be inferred from header presence, ciphertext lengths, and delivery behavior.
Visible by design	Protocol version, total frame length, delivery timing, transport endpoints, ratchet DH public key, ML-KEM ciphertext, and advertised KEM public key.	Required for routing or state synchronization. These fields are outside the metadata-confidentiality claim and are left to transport padding, sender-hiding delivery, or anonymity systems.

Lemma 4.2 (Two-Layer AEAD Independence)**.** *For a ratchet epoch, the chain key ck_e and metadata key mdk_e are disjoint segments of a jointly pseudorandom HKDF output. The payload message key $mk_{e,n}$ is later derived from ck_e under the* "Aura-Msg" *domain label. Given only $mk_{e,n}$ and not the root or metadata key material, the adversary obtains no information about mdk_e under the HKDF PRF assumption and the one-way chain derivation.*

Nonce structure. Payload nonces use a structured 12-byte layout:

$$nonce = prefix[8] \parallel counter_{\mathrm{LE}}[2] \parallel msg\_index_{\mathrm{LE}}[2]. \tag{13}$$

The prefix is sampled by the CSPRNG when the send nonce generator is created or reset at a ratchet boundary, the counter is monotonically increasing,

and the message index provides per-chain addressability. Metadata encryption uses an independent random 12-byte header nonce. A ratchet is required before the negotiated message-chain limit is exceeded or either 16-bit field is exhausted. The construction also exposes a nonce-budget warning threshold for operational monitoring.

Identity binding hash. Every AEAD additional-data construction includes an identity binding hash:

$$ibh = \text{SHA-256}(\text{“Aura-Identity-Binding”} \,\|\, \text{sort}(ed_I^{pub}, ed_R^{pub}) \,\|\, \text{sort}(IK_I^{pub}, IK_R^{pub})). \tag{14}$$

where both Ed25519 and X25519 public key pairs are lexicographically sorted before concatenation. This binds AEAD data to the unordered pair of identities. Role separation is provided by the handshake transcript, key-confirmation direction labels, and initiator/responder chain-key assignment.

4.5. Replay Protection

Replay protection is achieved through a bounded nonce cache:

- A HashSet of seen payload nonces, bounded to $W = 2{,}048$ entries.
- Nonces embed a monotonic counter in bytes [8:10]. When the cache is full, the oldest observed nonce entries are evicted first.
- Duplicate nonces are immediately rejected before payload decryption.

4.6. State Persistence

Session state is serialized via Protocol Buffers and protected by two mechanisms:

- **State integrity HMAC:** An HMAC-SHA256 tag computed over the serialized state (key derived from the root key via HKDF) authenticates serialized-state integrity. Rollback freshness requires the external monotonic counter used by sealed-state import/export.
- **Sealed export:** For persistent storage, the state is encrypted using a double-envelope construction: a random data encryption key (DEK) encrypts the state, and the DEK is encrypted under a caller-provided key encryption key (KEK), both using AES-256-GCM-SIV.

5. Security Model

The security model is inspired by the eCK family of AKE models [34, 33] and adapted to the hybrid post-quantum setting. Its oracle and freshness rules differ from standard eCK formulations. The claims below therefore apply only to the model defined here.

Definition 5.1 (Parties and Sessions)**.** There are at most N parties, each holding an X25519 identity keypair IK_i, an Ed25519 signing keypair, an ML-KEM-768 keypair (kpk_i, ksk_i), rotating signed pre-keys, and one-time pre-keys. A session π_i^s denotes a protocol run by party P_i with intended partner P_j.

Definition 5.2 (Adversary Oracles)**.** The adversary $\mathcal{A}$ has access to the following oracles:

- $\mathsf{Send}(\pi_i^s, m)$: Deliver message m to session π_i^s and obtain the response.
- $\mathsf{Corrupt}(P_i)$: Obtain all long-term keys of P_i (ik_i, Ed25519 signing key, ksk_i).
- $\mathsf{RevealSPK}(P_i, spk\_id)$: Obtain the private scalar for a selected signed pre-key.
- $\mathsf{RevealOPK}(P_i, opk\_id)$: Obtain the private scalar for an unused one-time pre-key. Consumed OPKs are modeled as deleted.
- $\mathsf{RevealState}(\pi_i^s)$: Obtain the full session state (root key, chain keys, metadata keys, DH/ML-KEM private keys, nonce state).
- $\mathsf{RevealSessionKey}(\pi_i^s)$: Obtain the current root key rk_e.
- $\mathsf{Test}(\pi_i^s)$: Challenge oracle. Flip coin $b \xleftarrow{\$} \{0,1\}$. Return rk_e if $b = 0$ and a uniformly random key if $b = 1$. At most one Test query is allowed.

Definition 5.3 (Partnering)**.** Sessions π_i^s and π_j^t are *partnered* if they share the same transcript hash th.

Definition 5.4 (Freshness)**.** A session π_i^s with partner π_j^t is *fresh* if the adversary has not:

1. Queried $\mathsf{RevealSessionKey}$ or $\mathsf{RevealState}$ on either π_i^s or π_j^t before Test.

2. Queried both $\mathsf{Corrupt}(P_i)$ and $\mathsf{RevealState}(\pi_i^s)$ before the tested session completed.
3. Queried $\mathsf{Corrupt}(P_j)$ before session π_j^t was created and no OPK was used.
4. Queried $\mathsf{RevealSPK}(P_j, spk\_id)$ for the responder signed pre-key used in the tested session before Test and no OPK was used and deleted.
5. Queried $\mathsf{RevealOPK}(P_j, opk\_id)$ for the OPK used in the tested session before it was consumed.

Definition 5.5 (Aura AKE Security)**.** Aura Protocol is secure in the model above if for all PPT adversaries $\mathcal{A}$:

$$\mathbf{Adv}^{\mathrm{AKE}}_{\mathrm{AURA}}(\mathcal{A}) = \left|\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}\right| \leq \mathrm{negl}(\lambda), \tag{15}$$

where $\mathcal{A}$ wins by correctly guessing b for a fresh tested session. This game measures key indistinguishability for fresh partnered sessions. The correspondence properties used for peer authentication are stated separately.

Definition 5.6 (Forward Secrecy)**.** If an adversary learns the parties' long-term identity keys and long-term KEM decapsulation keys at time t, the adversary still cannot decrypt messages sent before t in sessions whose key agreement completed before the compromise. This claim requires that either the responder signed-pre-key private scalar remains undisclosed or a consumed OPK private scalar was used and deleted.

Definition 5.7 (Post-Compromise Security [2])**.** After full state compromise at epoch e via $\mathsf{RevealState}$, messages at epoch $\geq e + k$ are secure, provided the adversary does not compromise new keys generated after epoch e. The analysis distinguishes $k = 1$ (classical PCS) from $k = 2$ (hybrid PCS).

6. Security Analysis

This section gives six reduction-style security claims for Aura Protocol. Each claim is accompanied by an argument sketch in the custom model of Section 5. These sketches identify the relevant assumptions and loss terms, but they are not substitutes for complete game-by-game reductions.

6.1. Hybrid Root-Key Pseudorandomness

Security claim 6.1 (Hybrid Root-Key Pseudorandomness)**.** *If* ML-KEM-768 *is* IND-CCA2 *secure* ***or*** X25519 *satisfies* Gap-CDH *in the* ROM*, then the Aura Protocol hybrid combiner*

$$rk_0 = \mathrm{HKDF}(ss_{ky},\ 32,\ cs,\ \text{“Aura-Hybrid-X3DH”}) \tag{16}$$

outputs a pseudorandom root key under the additional assumption in Definition 3.5. The argument gives two alternative reduction paths:

$$\mathbf{Adv}^{\mathrm{rk}}_{\mathrm{Hybrid}}(\mathcal{A}) \le \mathbf{Adv}^{\text{IND-CCA2}}_{\text{ML-KEM-768}}(\mathcal{B}_1) + \mathbf{Adv}^{\text{two-source PRF}}_{\mathrm{HKDF}}(\mathcal{B}_3) + \frac{q_H}{2^{256}}, \tag{17}$$

$$\mathbf{Adv}^{\mathrm{rk}}_{\mathrm{Hybrid}}(\mathcal{A}) \le 4 \cdot \mathbf{Adv}^{\text{Gap-CDH}}_{\text{X25519}}(\mathcal{B}_2) + \mathbf{Adv}^{\text{two-source PRF}}_{\mathrm{HKDF}}(\mathcal{B}_3) + \frac{q_H}{2^{256}}. \tag{18}$$

where q_H is the number of random-oracle queries. The classical path models the inner “Aura-X3DH” *extraction in the random-oracle model. The factor 4 accounts for selecting the DH term used by that path.*

Argument sketch. There are two reduction paths. In the KEM path, the argument replaces the ML-KEM shared secret ss_{ky} with a uniformly random value, reducing the distinguishing advantage to $\mathbf{Adv}^{\text{IND-CCA2}}_{\text{ML-KEM-768}}(\mathcal{B}_1)$. In the classical path, it replaces the classical pre-key shared secret cs with a random value via the Gap-CDH oracle. The factor 4 accounts for selecting one of the four DH pairs $dh_1, \ldots, dh_4$. Either path leaves at least one HKDF input uniformly random, so the two-source PRF property makes rk_0 indistinguishable from random. The two cases give Equations (17) and (18). The $q_H/2^{256}$ term bounds random oracle collisions. □

6.2. AKE Security

Security claim 6.2 (AKE Security in the Modified Pre-Key Leakage Model)**.** *Aura Protocol's hybrid pre-key agreement satisfies the AKE security definition above. For any* PPT *adversary $\mathcal{A}$ making at most q_s session queries, define the two alternative combiner-path losses*

$$\Delta_{\mathrm{KEM}} = \mathbf{Adv}^{\text{IND-CCA2}}_{\text{ML-KEM-768}}(\mathcal{B}_2) + \mathbf{Adv}^{\text{two-source PRF}}_{\mathrm{HKDF}}(\mathcal{B}_3),$$
$$\Delta_{\mathrm{DH}} = 4 \cdot \mathbf{Adv}^{\text{Gap-CDH}}_{\text{X25519}}(\mathcal{B}_1) + \mathbf{Adv}^{\text{two-source PRF}}_{\mathrm{HKDF}}(\mathcal{B}_3).$$

For either $X \in \{\mathrm{KEM}, \mathrm{DH}\}$ whose assumptions hold, the corresponding reduction path gives

$$\mathbf{Adv}^{\mathrm{AKE}}_{\mathrm{AURA}}(\mathcal{A}) \leq q_s \cdot \Big(\Delta_X + 2 \cdot \mathbf{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}}(\mathcal{B}_4) + 3 \cdot \mathbf{Adv}^{\text{SUF-CMA}}_{\mathrm{Ed25519}}(\mathcal{B}_5)\Big) + \frac{q_s^2}{2^{128}} + \frac{q_H}{2^{256}}. \tag{19}$$

No minimum of the two reduction advantages is asserted.

Argument sketch. The reduction guesses the tested session, losing a factor q_s. It then uses one of the two combiner paths in security claim 6.1: either replace the ML-KEM shared secret by random, or replace the surviving pre-key DH contribution by random. HKDF then yields a pseudorandom root key through the two-source PRF property. Finally, the reduction replaces the key-confirmation MACs with random tags, reducing to PRF security of HMAC. The factor 2 accounts for mac_I and mac_R. A successful identity or pre-key substitution must forge one of the initiator identity-binding, responder identity-binding, or responder signed-pre-key signatures. This gives the three SUF-CMA terms. The term $q_s^2/2^{128}$ bounds 128-bit session ID collisions. The signature term is classical. The claim does not provide post-quantum active authentication once Ed25519 is broken. □

Lemma 6.3 (Bilateral Key Confirmation Prevents UKS)**.** *Assume that the parties begin with verified identity keys and use the canonical transcript encoding. An adversary who does not know rk_0 cannot forge a valid key-confirmation MAC. The keys kc_I and kc_R are derived from rk_0 under distinct HKDF labels, and each MAC covers the transcript hash that binds both identities and the ephemeral key material.*

Lemma 6.4 (Domain Separation)**.** *Let $\mathcal{L}_{\mathrm{HKDF}}$ be the set of labels used as HKDF info strings in the construction. The labels in $\mathcal{L}_{\mathrm{HKDF}}$ are pairwise distinct and are disjoint from labels used for transcript hashing, identity binding, signatures, and auxiliary validation paths. Thus distinct HKDF invocations are modeled as distinct oracle inputs in the* ROM *and as distinct PRF inputs in the standard-model analysis.*

6.3. Forward Secrecy

Security claim 6.5 (Forward Secrecy)**.** *For any* PPT *adversary $\mathcal{A}$ who obtains the long-term identity and KEM keys at time t, and for sessions*

whose responder signed-pre-key private scalar remains undisclosed or whose consumed OPK private scalar has already been deleted:

$$\mathbf{Adv}^{\mathrm{FS}}_{\mathrm{AURA}}(\mathcal{A}) \leq \mathbf{Adv}^{\text{Gap-CDH}}_{\mathrm{X25519}}(\mathcal{B}_1) + \mathbf{Adv}^{\text{two-source PRF}}_{\mathrm{HKDF}}(\mathcal{B}_2). \tag{20}$$

This is a classical forward-secrecy bound under later disclosure of long-term KEM secret keys. If both the responder SPK private scalar and KEM secret key are later disclosed, and no deleted OPK contributed to the transcript, this classical FS claim does not apply. The initial KEM ciphertext gives harvest-now-decrypt-later protection only while the corresponding KEM secret key is not later disclosed. This is a fixed-session bound. A multi-session game incurs the corresponding session-selection loss.

Argument sketch. In the first hybrid transition, security comes from a DH term whose private scalar is unavailable after compromise: dh_3 if the responder SPK private scalar is not revealed, or dh_4 if a consumed OPK was used and deleted. The transcript-visible term $dh_2 = \mathrm{X25519}(ek_I, IK_R^{pub})$ is not counted after ik_R disclosure, because it can be recomputed from IK_R and EK_I^{pub}. The surviving classical shared secret is used as the HKDF salt, so the two-source PRF property makes the root key pseudorandom. The initial KEM term is intentionally not counted in this FS bound: after later disclosure of ksk_R, a recorded KEM ciphertext can be decapsulated. The KEM contribution in this setting is HNDL protection without later KEM secret-key disclosure, and PCS after fresh ratchet KEM keys appear. □

Lemma 6.6 (Ephemeral Key Erasure)**.** *Temporary DH outputs, KEM shared secrets, and intermediate key-derivation buffers are assumed to be erased immediately after the derivation that consumes them. A ratchet private key that remains part of the live session state is not treated as erased until a later ratchet transition replaces it. The implementation evidence in Section 8 addresses this erasure assumption at the level of executable state management.*

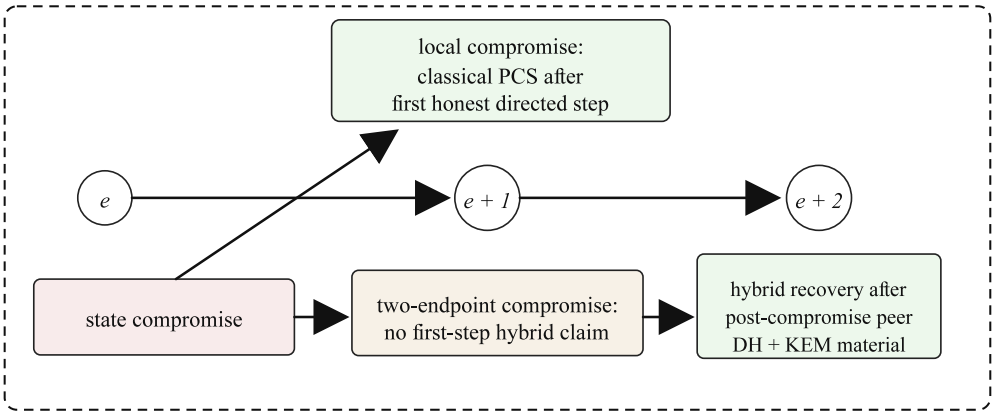

Figure 7: Recovery conditions for local versus two-endpoint state compromise. Recovery requires a later honest directed ratchet step.

Security claim 6.7 (Post-Compromise Security)**.** *If the adversary compromises session state at epoch e via* RevealState *and then stops learning post-compromise secrets:*
(a) Local classical PCS (1-step): *If the compromise is local to the sender endpoint, or more generally the peer DH private scalar used by the next directed DH remains undisclosed, messages after the first honest directed ratchet step are secure:*

$$\mathbf{Adv}^{\text{PCS-1}}_{\text{AURA}}(\mathcal{A}) \leq \mathbf{Adv}^{\text{Gap-CDH}}_{\text{X25519}}(\mathcal{B}_1) + \mathbf{Adv}^{\text{split-input PRF}}_{\text{HKDF}}(\mathcal{B}_2). \tag{21}$$

(b) Two-endpoint hybrid recovery: *If both endpoints' epoch-e DH and KEM ratchet secrets are exposed, no first-step PCS is claimed. Messages after the next directed step that uses post-compromise peer DH and KEM public keys are secure. The DH and KEM reduction paths respectively give*

$$\mathbf{Adv}^{\text{PCS-2}}_{\text{AURA}}(\mathcal{A}) \leq \mathbf{Adv}^{\text{Gap-CDH}}_{\text{X25519}}(\mathcal{B}_1) + \mathbf{Adv}^{\text{split-input PRF}}_{\text{HKDF}}(\mathcal{B}_2), \tag{22}$$

$$\mathbf{Adv}^{\text{PCS-2}}_{\text{AURA}}(\mathcal{A}) \leq \mathbf{Adv}^{\text{IND-CCA2}}_{\text{ML-KEM-768}}(\mathcal{B}_3) + \mathbf{Adv}^{\text{split-input PRF}}_{\text{HKDF}}(\mathcal{B}_2). \tag{23}$$

These are fixed-transition bounds. A game in which the reduction must select one transition among many adds the corresponding selection loss.

Argument sketch. The key subtlety is that the first post-compromise directed step uses the peer's epoch-e DH and KEM public keys. If the corresponding

peer DH private scalar was also revealed, the adversary can compute that first DH contribution from the sender's new public key. The claim therefore separates local state compromise from true two-endpoint state compromise.

Part (a): At the first honest directed step, the honest party generates a fresh DH keypair (d_{e+1}, D_{e+1}) unknown to the adversary. If the peer DH private scalar for D_{peer} was not exposed, the DH shared secret is fresh. The KEM encapsulation at that step uses the old kpk_e^{peer} (possibly compromised), so KEM does not contribute security. By Gap-CDH hardness and the split-input PRF property, the new root key is pseudorandom even though the previous root and the KEM contribution may be known in the local-compromise model.

Part (b): During the first honest directed step, the honest party also generated a fresh DH keypair and ML-KEM keypair (kpk_{e+1}, ksk_{e+1}) and advertised their public components. At the next directed step that uses these post-compromise public keys, the peer computes DH against the fresh DH public key and encapsulates against kpk_{e+1}, whose secret key was generated after compromise. Either the fresh DH contribution or the fresh KEM shared secret can serve as the fresh component of the concatenated ratchet input. The split-input PRF property gives pseudorandomness of the next root key even when the preceding root is already known.

Lemma 6.8 (Per-Direction Ratchet)**.** *Aura Protocol's per-direction ratcheting records, after each accepted inbound message, that the next outbound message must carry a fresh hybrid ratchet header. Thus every direction change triggers a DH/KEM refresh. This makes the recovery point explicit: after local state compromise, classical PCS starts at the next honest ratchet step. After two-endpoint state compromise, recovery is delayed until a directed step uses post-compromise peer DH and KEM material as described in security claim 6.7.*

6.5. Message Confidentiality and Integrity

Security claim 6.9 (Message Security)**.** *For fresh partnered sessions under non-adversary-established root keys, and under* MRAE *security of* AES-256-GCM-SIV *and* PRF *security of the chain KDF, each message key $mk_{e,n}$ is indistinguishable from random and each ciphertext provides*

IND-CPA + INT-CTXT *security over* q_m *total encrypted message attempts:*

$$\begin{aligned}\mathbf{Adv}^{\mathrm{Conf}}_{\mathrm{AURA}}(\mathcal{A}) \leq \mathbf{Adv}^{\mathrm{AKE}}_{\mathrm{AURA}}(\mathcal{B}_0) + q_m \cdot \mathbf{Adv}^{\mathrm{PRF}}_{\mathrm{chain}}(\mathcal{B}_1) \\ + q_m \cdot \mathbf{Adv}^{\mathrm{MRAE}}_{\text{AES-256-GCM-SIV}}(\mathcal{B}_2) + \frac{q_m^2}{2^{96}}.\end{aligned} \tag{24}$$

Argument sketch. The argument first replaces the relevant root key with a random value, using security claims 6.2, 6.5 and 6.7. It then applies a hybrid argument over chain indices, replacing each $mk_{e,n}$ with a random key by PRF security of the chain KDF. The last step invokes MRAE security. With random per-message keys, AES-256-GCM-SIV provides IND-CPA + INT-CTXT. Payload nonces are unique within a chain by construction. The term $q_m^2/2^{96}$ conservatively bounds collisions of random metadata header nonces under the same metadata key. □

Lemma 6.10 (GCM-SIV Nonce-Misuse Resistance)**.** AES-256-GCM-SIV *provides full* IND-CPA + INT-CTXT *under unique nonces, and degrades to deterministic authenticated encryption under nonce reuse: repeated key/nonce/AAD/plaintext tuples expose equality patterns, while authenticity remains bounded by the* MRAE *notion. Aura Protocol's monotonic counters are the primary mechanism for nonce uniqueness. The use of GCM-SIV provides residual authenticity and deterministic leakage bounds if a nonce is nevertheless reused.*

6.6. Replay Resistance

Security claim 6.11 (Replay Resistance)**.** *Assume continuous receiver state, erased used message keys, and no accepted sealed-state rollback. Exact replay acceptance is zero in this receiver-state model. For modified replays:*

$$\mathbf{Adv}^{\mathrm{Replay}}_{\mathrm{AURA}}(\mathcal{A}) \leq \mathbf{Adv}^{\mathrm{INT\text{-}CTXT}}_{\text{AES-256-GCM-SIV}}(\mathcal{B}). \tag{25}$$

Argument sketch. Exact replays are caught by the nonce cache while the entry is retained. If the entry has been evicted, the stale message key is no longer derivable from the current chain state, so the same ciphertext still cannot be accepted. Modified replays require forging an AEAD tag and reduce to INT-CTXT. Crash recovery is a separate state-continuity assumption enforced by sealed-state counters. Without durable state, replay resistance is a system-level contract rather than a pure AEAD property. □

Table 5: Assumptions and indicative security levels.

Component or claim	Basis	Interpretation
X25519 / Gap-CDH	Curve25519 [30]	Approximately 128 classical bits
ML-KEM-768 / Module-LWE	FIPS 203, category 3 [6]	NIST Level 3
AES-256-GCM-SIV / MRAE	RFC 8452 [28]	128-bit authentication tag
HKDF-SHA256	RFC 5869 plus Definitions 3.5 and 3.6	Explicit handshake and ratchet assumptions
HMAC-SHA256 / PRF	FIPS 198-1 [35]	PRF assumption, 256-bit output
Ed25519 / SUF-CMA	EdDSA [36]	Approximately 128 classical bits
Hybrid combiner (security claim 6.1)	Alternative DH or KEM path	Limited by the selected path
AKE security (security claim 6.2)	AKE reduction	At most 98 classical bits when $q_s \leq 2^{30}$
Forward secrecy (security claim 6.5)	Gap-CDH with erased DH ephemeral	Fixed-session reduction
PCS, classical (Equation (21))	Gap-CDH and split-input KDF, one transition	Fixed-transition reduction
PCS, hybrid (Equations (22) and (23))	Alternative DH or KEM path with split-input KDF	Fixed-transition reduction
Message security (security claim 6.9)	Root, chain-KDF, and MRAE terms	Inherits all component losses

6.7. Concrete Security Bounds

The session-selection factor q_s is the largest explicit loss in security claim 6.2. For $q_s \leq 2^{30}$, a classical 128-bit component gives an upper target of 98 bits before the remaining reduction terms are added. ML-KEM-768 is treated as a NIST category-3 KEM. Core-SVP estimates are estimator-dependent and are not protocol constants. The FS and PCS expressions are stated for a fixed session or transition. Their multi-session forms require an additional selection loss.

7. Formal Verification

The reduction-style arguments are accompanied by scoped symbolic analyses in two tools: Tamarin Prover [23] for trace-based reasoning and ProVerif [24] for Horn-clause analysis. These analyses provide bounded assurance for specific abstractions rather than a machine proof of the full executable protocol. Within the stated scope, the Tamarin models verify ten lemmas. The active ProVerif model discharges four queries. Three cover the primary handshake and message claims, and one checks honest-message correspondence under matching endpoints and root state. Injective authentication and raw-KEM secrecy after KEM-secret-key disclosure remain documented stress obligations. Broad unpartnered integrity is not claimed.

7.1. Tamarin Prover

7.1.1. Handshake Model

The handshake model uses Tamarin's built-in Diffie–Hellman equational theory and represents ML-KEM-768 by symbolic public-key generation, encapsulation, and decapsulation equations:

$$\mathrm{KEMPK}(sk) \to kpk, \tag{26}$$

$$\mathrm{KEMEncaps}(ss, kpk) \to ct, \tag{27}$$

$$\mathrm{KEMDecaps}(ct, sk) = ss. \tag{28}$$

Key design decisions.

- Only 2 of the 4 pre-key DH operations (dh_1 and dh_2) are modeled. This keeps source reasoning terminating while preserving a conservative secrecy abstraction. It does not represent every transcript binding path in the executable construction.
- Party key material is represented by one persistent key fact, which avoids artificial cross-instance mismatches caused by separately modeled long-term facts.
- The symbolic authentication model treats the responder KEM public key as a trusted bundle field. This is stronger than validation plus transcript/key-confirmation binding. KEM-key substitution is therefore handled by Lemma 4.1 and by executable tamper checks.

Table 6: Scope of mechanized and executable evidence.

Evidence source	In scope	Explicitly out of scope
Reduction-style claims	Hybrid root pseudorandomness, AKE, forward secrecy, PCS delay, message security, and replay resistance under stated assumptions.	Byte parser correctness, platform memory behavior, key-directory policy, and multi-device or group semantics.
Tamarin handshake model	Six lemmas for a reduced handshake abstraction with symbolic ML-KEM and selected DH structure.	Full four-DH source saturation, directory-side KEM substitution traces, and executable serialization.
Tamarin ratchet model	Four lemmas for a terminal one-step ratchet abstraction and sender-compromise recovery shape.	Unbounded asynchronous ratchet execution and the full two-endpoint hybrid recovery trace.
ProVerif model	Four active queries discharged: three primary handshake/message-path queries and one scoped honest-message correspondence check.	Injective authentication, broad unpartnered integrity, and raw KEM-secret secrecy after KEM-SK disclosure.
Executable validation	Parsing, rollback, replay, sealed-state continuity, malformed-input rejection, and deterministic vectors.	Cryptographic reductions, symbolic proof obligations, and deployment claims outside the two-party state machine.

Threat model. The Dolev–Yao adversary has full network control. Two compromise rules model different adversary capabilities:

- **Classical compromise:** Reveals X25519 identity and signed pre-key private keys (models side-channel or CRQC).
- **Full compromise:** Additionally reveals the ML-KEM secret key (total break).

Table 7: Tamarin handshake model: verified properties.

#	Property checked	Meaning
1	Hybrid root secrecy	The root secret is hidden unless the modeled compromise condition occurs.
2	Initiator authentication	Initiator completion implies a corresponding responder execution.
3	Responder authentication	Responder completion implies a corresponding initiator execution.
4	Hybrid forward secrecy	Classical-only compromise does not reveal the established session key.
5	Key confirmation	Partnered sessions derive identical root keys.
6	Reachability	An honest execution can complete.

Verified handshake properties. The fourth property is the hybrid secrecy statement in the handshake abstraction: if the adversary learns the session key k, then either a party was compromised before the session, enabling active impersonation, or full compromise including the KEM secret key occurred. Classical-only compromise after the session does not reveal the key in that model.

The reported run invokes Tamarin with a 60-second derivation-check budget. On the evaluation host, Tamarin's five-second default budget produces a timeout warning even though all six proof searches finish. Increasing that budget to 60 seconds completes all well-formedness checks and all six lemmas without the warning.

7.1.2. Ratchet Model

The ratchet model abstracts DH as a classical KEM and uses a terminal single-step transition. This avoids non-termination caused by combining state-loop rules with the DH equational theory. The model verifies the decomposed PCS and key-agreement shape for that step. The unbounded ratchet PCS statement remains a reduction claim, and augmented KEM-public-key binding is handled by the HKDF lemma and executable tamper checks.

Key design decisions.

- Ratchet chains combined with the DH equational theory lead to non-termination. The model therefore represents a single ratchet step with compromise fixed at setup time.

- Three setup variants (honest, sender-compromised, receiver-compromised) replace runtime compromise rules, giving the adversary different levels of initial knowledge.
- A linear sent-ciphertext fact binds sender and receiver within the same session, so the receiver processes only ciphertexts represented as sent in the trace.

Table 8: Tamarin ratchet model: verified properties.

#	Property checked	Meaning
1	Sender-compromise recovery	After sender-state compromise, a fresh ratchet key is secret unless the receiver is also compromised.
2	Ratchet-key secrecy	The ratchet key is secret absent sender or receiver compromise.
3	Ratchet key agreement	Both parties derive the same root key after the modeled ratchet step.
4	Reachability	An honest ratchet step can complete.

Verified ratchet properties. The first property captures sender-root compromise in a terminal one-step ratchet abstraction: the adversary knows rk_e, and a fresh hybrid ratchet step restores secrecy unless the receiver-side DH/KEM secrets are also compromised. The two-endpoint two-step hybrid PCS claim remains a reduction result rather than a direct machine-checked consequence of this model.

7.2. ProVerif

The active ProVerif model covers the handshake plus one symbolic message path. It models all four pre-key DH operations, Ed25519 signature verification, and the initial chain KDF over one public Dolev–Yao channel with unbounded session replication. Ratchet recovery is outside the active ProVerif process and is analyzed by the Tamarin ratchet abstraction and the PCS claim.

Trust model. Identity keys are registered in a symbolic table, modeling out-of-band key verification such as trust-on-first-use or safety-number comparison.

Phase-based forward secrecy. Phase 0 represents honest execution. A later phase reveals long-term keys only for auxiliary checks outside the active proof-scope query set. The reported queries cover no-compromise secrecy, non-injective authentication, and challenge-message integrity under the same derived root.

Table 9: ProVerif model: scoped query status.

#	Property checked	Meaning	Status
Q1	KEM shared-secret secrecy	Phase-0 KEM secret feeding rk_0 hidden from adversary	✓
Q2	Authentication (non-inj.)	Initiator complete ⇒ responder accepted	✓
Q3	Authentication (inj.)	Injective correspondence for Q2	n/c
Q4	Message secrecy	Phase-0 encrypted payload secret from adversary	✓
Q5	Honest-message correspondence	Matching challenge plaintext, endpoints, and root on send and receive	✓
Q6	Raw KEM secrecy after KEM-SK reveal	Phase-1 KEM-SK disclosure does not reveal the raw phase-0 KEM secret	n/c

ProVerif query set.

Auxiliary queries outside the claim boundary. Q3 is documented in the model but disabled in the default artifact because the four-DH injective-authentication query does not terminate within the artifact budget. Q5 is intentionally broad and negative: an active adversary can create its own session and deliver its own plaintext, so that query is not the partnered-session integrity property used in security claim 6.9. The active model instead checks honest-message correspondence, requiring the same challenge plaintext, endpoints, and derived root on send and receive. This does not establish broad unpartnered integrity. Q6 is false by decapsulation semantics after the responder's KEM secret key is revealed. The forward-secrecy claim concerns the hybrid root under erased DH ephemerals, not raw KEM-secret secrecy after KEM-SK disclosure. Ratchet PCS is split: the terminal one-step abstraction is covered by Tamarin, while the two-endpoint hybrid recovery claim remains reduction-style.

7.3. Complementarity of Tools

The two tools play complementary roles. Tamarin provides precise DH reasoning but requires abstractions to avoid non-termination. ProVerif handles unbounded handshake/message sessions with all four DH operations and helps identify overbroad query formulations. The machine-checked evidence therefore supports the scoped secrecy and correspondence claims stated above, while injective authentication, broad unpartnered integrity, and raw-KEM-after-reveal secrecy remain outside the reported machine-checked boundary. Ratchet secrecy and PCS evidence is split: Tamarin checks a terminal one-step sender-compromise abstraction, while the full local-vs. two-endpoint PCS delay remains a reduction-style claim supported by executable state-machine validation.

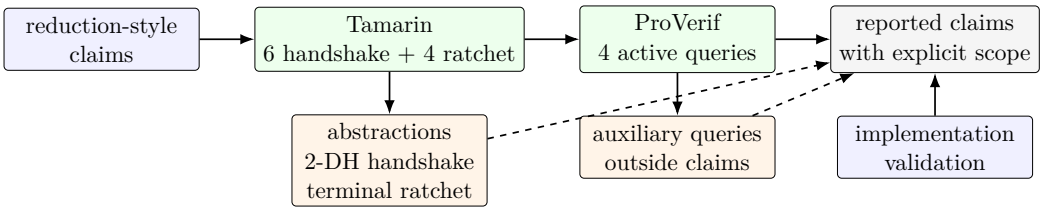

Figure 8: Formal-evidence map. The manuscript combines reduction-style arguments, scoped symbolic models, and implementation validation. Abstractions and auxiliary queries are shown separately from the claims counted as verified evidence.

8. Implementation

The evaluated implementation follows the protocol decomposition used in the preceding sections. Its role in this paper is not to serve as a proof object, but to demonstrate that the mathematical state machine can be realized with explicit parsing, rejection, erasure, replay, and persistence boundaries. The implementation is organized around the following architectural responsibilities.

Table 10: Implementation architecture.

Layer	Responsibility
Core definitions	Versioning, protocol constants, error semantics, and bounded input sizes.
Cryptographic layer	Primitive instantiations for X25519, Ed25519, ML-KEM-768, HKDF, HMAC, AES-256-GCM-SIV, secure memory, and validation routines.
Identity and pre-key layer	Identity generation, signed pre-key handling, one-time pre-key selection, and responder bundle construction.
Session protocol layer	Hybrid pre-key agreement, root/chain ratchets, envelope processing, replay windows, and sealed-state import/export.
Validation boundary	Public-key validation, malformed-input rejection, downgrade handling, timestamp normalization, and state-transition atomicity checks.
External interface layer	A narrow interface for creating identities, initiating sessions, encrypting/decrypting messages, and serializing state.

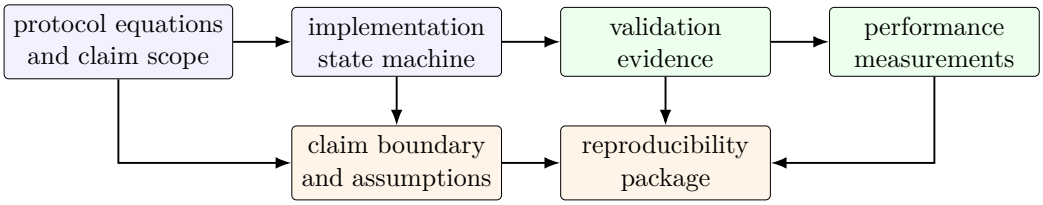

Figure 9: Implementation-evidence map. Executable validation and performance measurements are used to support the stated implementation scope. They do not replace the reduction-style claims or symbolic models.

8.1. Cryptographic Library Choices

Table 11: Concrete primitive choices in the reference implementation.

Primitive	Implementation note
X25519 and Ed25519	Constant-time curve operations with public-key validation and protected handling of private material where supported.
ML-KEM-768	FIPS 203 KEM used for initial and ratchet-level post-quantum contributions. Deterministic generation is restricted to reproducibility vectors.
AES-256-GCM-SIV	RFC 8452 misuse-resistant AEAD for payloads, metadata, and sealed state.
HKDF-SHA256 and HMAC-SHA256	RFC 5869 and RFC 2104 constructions with explicit domain-separated labels.
Serialization	Length-bounded Protocol Buffers encoding with a fixed maximum message size.
Secret handling	Automatic and explicit zeroization of temporary key material, with platform memory protection for long-lived secrets where supported.

8.2. Secure Memory Management

Key material is protected at multiple levels:

1. **Memory protection for long-lived secrets.** Long-lived DH and ML-KEM secret keys are stored through a memory-protection layer. Unix-like builds use a bounded, preallocated page-locked pool and request crash-dump exclusion where the platform provides it. Failure to establish or allocate from that pool is not silently downgraded to pageable memory. The iOS and Windows backends use heap storage with automatic zeroization because this Unix locking path is not selected on those targets.
2. **Scope-bound zeroization.** Intermediate key material, including HKDF extract outputs and chain-key copies, is wrapped so that it is zeroized when it leaves scope, including error paths.
3. **Immediate zeroization of consumed secrets.** DH shared secrets, hybrid input-keying buffers, classical shared secrets, and ML-KEM shared secrets are explicitly zeroized immediately after their consuming derivation.

4. **Timestamp rounding.** Nanosecond fields in message timestamps are set to zero to reduce device fingerprinting through timing data.

8.3. ML-KEM-768 Integration

The ML-KEM-768 integration has three protocol-relevant points that affect the validation boundary:

- **Self-testing.** Random ML-KEM key generation is followed by an immediate encapsulate/decapsulate round-trip with constant-time shared secret comparison. Deterministic seeded generation is reserved for reproducible vectors. The self-test detects KEM correctness failures, including randomness-source, implementation, or integration faults that cause encapsulation and decapsulation to disagree.

- **Deterministic vectors.** Deterministic KEM generation is used only to produce reproducible vectors from an explicit seed. Normal protocol execution uses fresh randomness and does not rely on global deterministic hooks.

- **Bandwidth overhead.** Each hybrid ratchet step carries 1,184 + 1,088 + 32 = 2,304 bytes of cryptographic material (ML-KEM public key + ciphertext + DH public key). Protocol Buffers field tags, length prefixes, the previous-chain-length value, and transport framing make the serialized increment slightly larger and value-dependent. For messaging applications with typical message sizes of 100–1,000 bytes, the raw cryptographic material alone represents an approximate 3–23× expansion per ratchet step.

9. Evaluation

9.1. Performance Benchmarks

Benchmarks are single-host Criterion measurements with 200 samples and 5-second measurement windows. They are hardware-dependent implementation-cost snapshots, not security assumptions. The reproducibility material records the platform, compiler and dependency context, raw measurement outputs, and the inputs used for the summarized results in Tables 12 to 14. A fixed 128-message, 256-byte traffic profile is also used to compare handshake-only post-quantum key exchange, sparse chain-boundary

rekeying, periodic chain-boundary rekeying, and per-reply post-quantum rekeying under the same measurement conditions.

The artifact manifest identifies the exact repository revision used for the reported measurements and records the dirty-worktree summary, OS/kernel, CPU summary, Rust compiler version, dependency lockfile hash, build-profile context, and raw Criterion output in `versions.txt`, `MANIFEST.txt`, and `SHA256SUMS`. The tables below therefore summarize one auditable host profile rather than a portable latency guarantee.

Table 12: Core operation latencies.

Operation	Latency
Identity creation (5 OPKs)	$\sim$1.5 ms
Full handshake (keygen + pre-key DH + ML-KEM + confirm)	$\sim$1.1 ms
Handshake only (pre-created identity)	$\sim$0.7 ms
Hybrid ratchet step (X25519 + ML-KEM-768)	$\sim$259 μs
Encrypt 256 B (same chain, no ratchet)	$\sim$17 μs
Decrypt 256 B (same chain, no ratchet)	$\sim$21 μs
ML-KEM-768 keygen	$\sim$80 μs
ML-KEM-768 encapsulate	$\sim$100 μs
ML-KEM-768 decapsulate	$\sim$90 μs
X25519 scalar multiplication	$\sim$50 μs
HKDF-SHA256 (32 B output)	$\sim$2 μs
Session export (sealed)	$\sim$30 μs
Session import (sealed)	$\sim$35 μs
Shamir split (3-of-5, 32 B)	$\sim$5 μs
Shamir reconstruct (3-of-5, 32 B)	$\sim$3 μs

Table 13: Steady-state envelope decrypt latency by payload size.

Payload Size	Decrypt Latency
64 B	∼35 µs
256 B	∼38 µs
1 KiB	∼45 µs
4 KiB	∼60 µs
16 KiB	∼120 µs
64 KiB	∼350 µs
256 KiB	∼1.2 ms
1 MiB	∼4.5 ms

Table 14: Ratchet overhead comparison.

Component	Latency
X25519 DH scalar multiplication (classical baseline)	∼50 µs
ML-KEM-768 encap + decap (PQ overhead)	∼190 µs
Full hybrid ratchet step (DH + ML-KEM + HKDF + encrypt)	∼259 µs
Alternating throughput (ratchet every message, 256 B)	∼280 µs/msg
Burst decrypt throughput (pre-generated envelopes, 256 B)	∼15 µs/msg

Analysis. In these measurements, a hybrid ratchet step costs about 259 µs, versus about 50 µs for X25519 alone. Steady-state same-direction burst decrypts measure about 15 µs/msg. Alternating 256-byte messages, which time encryption and decryption in the loop, measure about 280 µs/msg. The full handshake measures about 1.1 ms on the benchmark host.

9.2. Executable Validation

The pinned implementation snapshot contains 750 default test scenarios. Table 15 reports the 556 non-VoIP scenarios used in the manuscript's validation ledger. The excluded 194 VoIP scenarios exercise a separate media state machine. The reported set also contains engineering checks outside the formal two-party model. Only the checks mapped in Table 16 support claims about the two-party protocol.

Table 15: Non-VoIP executable validation reported for the pinned snapshot.

Category	Scenarios
Primitive and protocol checks	36
External-interface checks	71
End-to-end protocol-state scenarios	392
Adversarial and negative-session scenarios	40
Header, time, malformed-input, and property scenarios	13
Additional compatibility scenarios	4
Reported validation scenarios	**556**

Property-based validation. Six property-based checks generate random inputs across a range of parameters: message counts (1–50), payload sizes (0–4,096 bytes), direction patterns, Shamir SSS parameters, and message permutations. These complement the deterministic checks by exploring the input space more broadly.

Security-property coverage mapping. Table 16 maps each reduction-style security claim to executable checks that exercise the corresponding concrete invariants:

Table 16: Validation mapping: formal properties and implementation coverage.

Security Property	Implementation evidence
Hybrid combiner (security claim 6.1)	Initiator/responder agreement checks, deterministic-vector checks, and known-answer checks for hybrid derivation.
AKE security (security claim 6.2)	Full handshake agreement, bilateral key confirmation, malformed-bundle rejection, and reflection-rejection checks.
Forward secrecy (security claim 6.5)	Old-chain-key isolation and multi-step ratchet-recovery checks.
Post-compromise security (security claim 6.7)	Post-compromise ratchet recovery and delayed-delivery recovery checks across multiple ratchet steps.
Message security (security claim 6.9)	Envelope AEAD checks, message round trips, malformed-envelope rejection, and property-based AEAD/message round trips.
Replay resistance (security claim 6.11)	Duplicate-ciphertext rejection, persisted replay-window checks, and old-epoch rejection.

10. Operational Conditions for the Implementation

The claims in Section 6 concern an abstract two-party session protocol. The implementation checks in Sections 8 and 9 examine whether parsing, persistence, and error handling preserve the state assumptions used by those claims. The required operational conditions are stated below.

Reject without state movement. For syntactically invalid input, the required implementation semantics is a pure rejection:

$$\text{Handle}(S, C) = (\bot, S) \qquad \text{for every ciphertext or frame } C \notin \mathcal{L}_{\text{AuraProtocol}}. \tag{29}$$

Here $\mathcal{L}_{\text{AuraProtocol}}$ is the set of well-formed protocol frames with valid version tags, fixed-length X25519 and ML-KEM fields, bounded counters, and a syntactically present ratchet header whenever the message type requires one. The implementation checks those conditions before any secret state is advanced. In particular, malformed protobuf fields, unsupported versions, incorrect public-key lengths, oversized payloads, missing ratchet material, and failed AEAD authentication must not increment message counters, change

replay windows, cache partial keys, or rotate the root key. This invariant is what allows the formal model to treat malformed traffic as absence of an accepted event rather than as a third kind of state transition.

Atomic acceptance. For valid traffic, acceptance is modeled as a single transition from state S_i to state S_{i+1}. The implementation has to preserve the same abstraction even though it performs parsing, decryption, replay checks, chain-key updates, metadata-key updates, and sealed-state export as separate instructions. The required order is:

$$\text{Verify}(C, S_i) = 1 \implies \text{Persist}(S_{i+1}) \prec \text{Release}(M), \tag{30}$$

unless the caller provides an equivalent durable transaction boundary. If plaintext were released before replay state and chain state became durable, a process crash could allow the same ciphertext to be accepted again after restart. That would not contradict AEAD security or HKDF pseudorandomness. It would violate the state premise of the replay claim. This is why sealed state, the external monotonic counter, and idempotent restore handling are part of the protocol evidence rather than mere storage convenience.

Out-of-order delivery. Asynchronous messaging cannot rely on FIFO transport. A delayed message may arrive after a peer has already sent a new ratchet header, and an old ciphertext may be replayed after a sealed-state import. The implementation therefore binds skipped message keys and cached metadata keys to the epoch in which they were derived. Accepting an old but still admissible message must not roll the root key backward, reinterpret a metadata key under a later epoch, or make a previously rejected ciphertext valid in a new state. Bounded caches are part of the same invariant: they prevent an attacker from forcing unbounded retention of old chain material and make the replay window a concrete system parameter rather than an implicit infinite buffer.

Key-directory and downgrade behavior. The pre-key directory is not trusted to define the peer's identity. It can delay updates, replay an old signed prekey, omit a KEM public key, or serve different bundles to different clients. Aura Protocol's claim is therefore conditional on identity verification and on a policy decision that the peer is expected to support the hybrid profile. Under that policy, absence or substitution of KEM material is a visible protocol failure, not a silent fallback to a weaker transcript. This distinction matters

for the harvest-now-decrypt-later claim: the hybrid argument applies to sessions whose transcript actually contains the ML-KEM contribution specified in Section 4.2.

Why the tests are scoped. The 556 validation scenarios in Table 15 are reported only insofar as they exercise the invariants above: domain separation, ratchet atomicity, replay-window persistence, sealed-state freshness, malformed-input rejection, and negative session behavior. Other engineering checks do not enlarge the security claim. This keeps the empirical results aligned with the exact two-party protocol analyzed in Section 5. Broader product surfaces require their own state models before analogous claims can be made.

11. Comparison with Current Post-Quantum Messaging Baselines

For a 2026 comparison, PQXDH is no longer the sole baseline. It is a handshake-level post-quantum upgrade, whereas Signal SPQR/Triple Ratchet and Apple PQ3 move post-quantum material into the ongoing evolution of the session state. The table below separates where PQ entropy enters the protocol, how PCS is recovered after compromise, and what public evidence supports the design.

Table 17: Positioning of Aura Protocol against current post-quantum messaging baselines.

Protocol	Where PQ material is used	PCS after compromise	Public evidence	Implication for Aura Protocol
Signal X3DH/Double Ratchet [1, 10]	Not used	Classical PCS through the DH ratchet	Open specifications and academic analyses	Baseline FS/PCS model, but not quantum-resistant.
Signal PQXDH [8]	Initial handshake	No renewed PQ contribution after the handshake	Open specification	Handshake-only baseline for initial harvest-now-decrypt-later protection, not for PQ-PCS in a long-lived channel.
Signal SPQR/Triple Ratchet [9, 10, 11, 37]	Sparse PQ ratchet alongside the classical Double Ratchet	PQ-PCS through SCKA epochs governed by ML-KEM Braid state and chunk/codeword delivery	Open specifications, public SPQR reference material, and Signal engineering notes	Open Signal-family baseline for ongoing PQ state evolution.
Apple PQ3 [12, 13, 14]	Initial establishment and periodic in-band rekeying	Published analyses claim FS/PCS against classical and quantum adversaries	Public design description, reduction-style analysis, and Tamarin analysis. Closed implementation	Deployed industrial baseline for periodic in-band post-quantum rekeying.
Aura Protocol	Hybrid handshake and KEM rotation at directed ratchet boundaries	Classical recovery after the next honest directed step. Hybrid recovery after a step using a post-compromise KEM key	Six reduction-style claims, scoped Tamarin/ProVerif results, and executable validation	Explicit KEM-key binding, per-epoch metadata encryption, and state-machine validation boundaries.

Table 18: Compact comparison against nearest post-quantum messaging baselines.

Feature	PQXDH	SPQR	PQ3	**Aura Protocol**
PQ contribution after handshake	No	Yes	Yes	**Yes**
PQ update schedule	One-time KEM	Sparse PQ ratchet	Periodic in-band rekeying	**KEM rotation at ratchet boundaries**
PQ-material binding	Initial KEM/session transcript	Braid authenticator MACs over header/ciphertext state	Authenticated in-band PQ rekeying in public analyses	**Ratchet KEM public key in HKDF info, AEAD-confirmed by the next message**
Separate envelope-metadata AEAD	Not specified by PQXDH	Not the focus of public Triple Ratchet/SPQR specs	PQ3 discusses padding and delivery receipts, not this envelope-key split	**Separate per-epoch metadata key**
Reference implementation	Specification	Public SPQR specifications and reference material	Public design, closed implementation	**Open reference implementation with validation material**
Formalized PQ-PCS delay	No	SCKA epoch analysis	Published PQ3 analysis	**PCS boundary for local vs. two-endpoint compromise**

The comparison identifies Aura Protocol's specific design role. The claimed novelty is construction-level, not the use of ML-KEM in messaging by itself. Signal SPQR provides the closest Signal-family baseline for sparse post-quantum state evolution, whereas Apple PQ3 represents a deployed industrial design with periodic in-band rekeying. Within this landscape, Aura Protocol jointly analyzes a direction-triggered X25519/ML-KEM transition, HKDF binding of each advertised KEM key, rotating metadata keys, and an explicit local-versus-two-endpoint PCS boundary in one open state machine. The implementation and formal models make that construction independently inspectable.

11.1. Ratcheting Strategy

The relevant design choice is trigger granularity. Classical DH ratchets advance when a new peer DH public key is received [10], while sparse post-quantum schedules amortize KEM work across longer state epochs. Aura Protocol uses per-direction hybrid ratcheting. After every received message,

the next send triggers a full X25519 and ML-KEM transition. Each direction change therefore installs a new root key and limits a chain-key compromise to the affected direction and epoch. The KEM contribution is also renewed beyond the initial handshake. Following local compromise, classical PCS begins at the next honest transition. Following two-endpoint compromise, hybrid recovery waits until a directed transition uses peer DH and KEM material generated after the compromise.

Cost. Each hybrid ratchet step adds $\sim$259 μs of computation and 2,304 bytes of raw cryptographic material (1,184 bytes for the ML-KEM public key, 1,088 bytes for the ML-KEM ciphertext, and 32 bytes for the DH public key), plus serialization framing. For interactive messaging with alternating messages, the measured profile is $\sim$280 μs/msg. The $\sim$15 μs/msg burst number is a decrypt-only measurement over pre-generated same-direction envelopes.

11.2. PCS Recovery: 1-Step vs. 2-Step

Classical DH ratchets achieve PCS once an uncompromised DH contribution enters the root chain. Aura Protocol separates two cases. If only the local endpoint state is compromised, the next honest directed step gives 1-step *classical* PCS via Gap-CDH on fresh DH material. If both endpoints' epoch state is compromised, the first directed step may still use peer ratchet state exposed before recovery. Hybrid recovery is claimed only once a directed step uses peer DH and KEM material generated after the compromise. Security claim 6.7 formalizes these recovery conditions.

11.3. Identity Binding

Authenticated pre-key messaging protocols bind long-term identities through signed pre-key material, authenticated key-agreement transcripts, and associated data. Aura Protocol makes this binding explicit by deriving an identity-binding hash

$$ibh = \text{SHA-256}(\text{“Aura-Identity-Binding”} \,\|\, \text{sort}(\cdot))$$

and embedding it in every AEAD additional-data field. Including both identities prevents cross-session identity confusion. Lexicographic sorting gives a stable unordered peer-set value, while transcript and key-confirmation domains retain the initiator and responder roles. The dedicated domain label prevents cross-protocol reuse, and including the hash in both metadata and payload AAD applies the same binding to both encrypted layers.

12. Discussion

12.1. Bandwidth Overhead

The primary cost of per-ratchet hybrid key exchange is bandwidth. Each ratchet step carries 2,304 bytes of raw DH/KEM material before serialization framing, which is approximately a 23× increase for a message carrying approximately 100 bytes of application data. Bursts amortize this cost because only the first message in a direction carries the ratchet header. Later messages in the same epoch have no KEM header. The protocol assumes no compression of KEM fields because compression would require separate security and side-channel analysis. Replacing ML-KEM-768 with ML-KEM-512 would reduce the comparable header to 1,600 bytes (800 + 768 + 32), but would also lower the post-quantum target to NIST Level 1.

12.2. 2-Step Hybrid PCS

The 2-step hybrid PCS recovery follows from the directed KEM-public-key ratchet used in Aura Protocol. After a local compromise, the first honest directed step can restore classical security through a fresh DH contribution. After full two-endpoint state compromise, however, the peer DH and KEM state used by that first directed step may already be exposed, so no one-step recovery is claimed. Full hybrid recovery is restored on a later directed step that uses peer DH and KEM material generated after the compromise.

Within this KEM-public-key ratchet pattern, one-step hybrid recovery would require a different schedule, such as simultaneous KEM-key transport, an acknowledged key update, or another post-quantum SCKA design. The analysis does not claim that the two-step delay is a lower bound for other ratchet designs.

12.3. ProVerif Stress Checks

The ProVerif model discharges the primary proof-scope queries reported in Table 9, but the stress checks remain outside that count. Injective authentication exceeds the practical scope of the four-DH ProVerif model within the artifact budget. Broad unpartnered message integrity is false for the active trace in which an adversary establishes its own session and delivers its own plaintext. Raw KEM shared-secret secrecy after the corresponding KEM secret key is revealed is false by decapsulation semantics, and therefore is not the forward-secrecy property claimed here. The claimed properties are carried by the Tamarin authentication, secrecy, and PCS abstractions, the

reduction-style arguments, and the executable validation categories described above.

12.4. Kyber/ML-KEM Terminology

The protocol is described in terms of ML-KEM-768. Earlier Kyber terminology may appear in legacy implementation identifiers, but no Kyber-specific security claim is made here. The reference implementation is not a certified FIPS module. A certified-provider migration is intended to be confined to the KEM wrapper and validation boundary while preserving protocol messages and proof assumptions. The hybrid combiner's security relies on IND-CCA2 security of the KEM, which ML-KEM provides.

12.5. Operational Profiles

The intended operating point is asynchronous text messaging. In that setting, a full hybrid ratchet on direction change keeps the recovery rule simple while amortizing the 2.3 KB of raw ratchet material over bursts of messages sent in the same direction. After a local state compromise, the first honest ratchet step restores the classical DH contribution. After full two-endpoint state compromise, recovery is tied to a later directed step that uses post-compromise peer DH and KEM material.

Latency-constrained profiles with a hard per-packet budget are a separate design point. The handshake can still establish an authenticated hybrid secret, but the ratchet schedule would need its own timing, acknowledgement, and loss model. The two-party messaging claims should not be read as proofs for such a profile unless that profile carries its own state machine and recovery argument.

Long-lived desktop and server-side sessions put pressure on a different part of the design: sealed state. A valid encrypted state blob is not necessarily a fresh state blob. Backup restore, VM snapshots, and storage migration can replay an old but correctly authenticated state. The sealed-state counter is therefore part of the protocol integration rather than a serialization detail. Without an external monotonic value, AEAD and HMAC authenticate the blob but cannot tell a current state from an older valid one.

Multi-device accounts introduce another boundary. The current model treats a session as a two-party channel between two devices. If one user has several devices, the system has to decide whether to run separate pairwise sessions, fan out messages through a Sesame-like session manager, or introduce a separate multi-device state-management layer. These choices affect

replay windows, identity binding, device removal, and sealed-state freshness. They are deployment-level choices, and they should not be folded into the two-party claims.

12.6. Reproducibility and Traceability

The computational arguments, symbolic models, and executable checks answer different questions. The arguments state the primitive assumptions and security losses. Tamarin and ProVerif examine selected trace properties under documented abstractions. The executable scenarios cover serialization, nonce discipline, replay state, skipped-key handling, and rejection of malformed envelopes. Table 16 links these checks to the protocol properties they exercise without treating test success as a cryptographic reduction.

This traceability also defines the work required after a protocol change. A change to a key derivation must be reflected in the argument and symbolic models. A change to framing or persistence requires new executable checks. The same discipline makes comparison with Signal SPQR and Apple PQ3 concrete: Aura Protocol exposes ratchet state, KEM-key binding, metadata-key rotation, and sealed-state rollback handling in one reproducible package.

The accompanying reproducibility material is organized to rebuild the paper, rerun the validation suite, rerun the symbolic models, and reproduce the benchmark summaries. Deterministic vectors cover HKDF, AES-GCM-SIV, seeded ML-KEM public-key generation, master-key identity derivation, the full handshake transcript, the first post-handshake envelope, the first DH+KEM-ratchet envelope, and multi-epoch delayed-delivery envelopes. The archived material records tool versions, manifests, checksums, formal-model logs, paper-build logs, and benchmark summaries. These artifacts are structured so that independent reviewers can compare model assumptions with the implementation paths exercised by the validation suite.

Table 19: Ranked limitations and deployment effect.

Severity	Limitation	Deployment effect and treatment
High	Classical identity authentication.	Ed25519 identity and SPK signatures leave active impersonation outside the post-quantum claim once a CRQC can forge signatures. A migration to ML-DSA or another post-quantum signature scheme is required for post-quantum authentication. The ratchet analysis remains a confidentiality argument under the current classical-authentication assumptions.
High	Sealed-state freshness.	AEAD and state HMACs authenticate serialized state but cannot distinguish a current blob from an older valid blob. Rollback protection therefore requires an external monotonic counter or equivalent freshness anchor.
Medium	Key-directory and TOFU assumptions.	The protocol assumes identity verification and policy enforcement for the hybrid profile. Directory equivocation, stale bundles, and downgrade attempts are treated as deployment risks unless the client detects and rejects them.
Medium	Single-device two-party scope.	The analyzed model is one device talking to one device. Multi-device fan-out, Sesame-like session management, and group messaging require separate state machines and proofs.
Medium	Ratchet cost and bandwidth.	Per-direction KEM rotation carries 2.3 KB of raw cryptographic material plus serialization framing and adds about 259 μs of ratchet work on the measured host. Lower-frequency ratchet schedules may be attractive but need their own PCS and cost statements.
Medium	Symbolic-model abstraction.	Tamarin and ProVerif support scoped claims, not complete protocol verification. The executable validation suite checks concrete parsing and rollback behavior, but it does not replace reductions or platform audits.

13. Conclusion

Aura Protocol contributes a directed hybrid-ratchet state-transition method for continuing post-quantum confidentiality in a two-party secure-messaging channel. The construction-level novelty lies in coupling each

direction change to X25519/ML-KEM renewal, binding the advertised KEM public key into the derived state, and stating separate recovery conditions for local and two-endpoint compromise. Local recovery obtains a fresh classical contribution at the next honest transition. Hybrid recovery requires a later transition that uses peer DH and KEM material created after the compromise. The method also rotates a separate metadata key per ratchet epoch and uses nonce-misuse-resistant AEAD for metadata and payload.

Six reduction-style arguments state the computational assumptions behind the construction. Ten Tamarin lemmas, three primary ProVerif queries, and one scoped correspondence query examine reduced handshake, ratchet, secrecy, and correspondence properties. Executable validation covers parsing, persistence, replay handling, negative-state behavior, and sealed-state import. Together these results connect the abstract state transitions to the implemented two-party message path while preserving the stated limits of each analysis.

The evaluated implementation indicates that the design is practical for asynchronous messaging profiles. On the reported host, the full handshake takes approximately 1.1 ms, alternating 256-byte messages cost about 280 μs/msg, and steady-state same-direction decrypts cost about 15 μs/msg. The 2,304 bytes of raw ratchet key and ciphertext material are the dominant deployment cost for short messages. Post-quantum identity authentication, stronger state-freshness anchors, multi-device operation, group messaging, mobile-class measurements, and lower-frequency ratchet schedules remain outside the analyzed construction. Aura Protocol therefore establishes an implementation-backed design for continuing post-quantum confidentiality in a two-party stateful channel, with its recovery delay and operational cost stated quantitatively.

Data and Software Availability

The source code, validation material, formal models, reproducibility scripts, benchmark definitions, and manuscript sources are available at https://github.com/oleksandrmelnychenko/aura-protected-protocol-rs. The evaluated implementation snapshot is commit `b788e07`. The reproduction script records tool versions and produces `MANIFEST.txt`, `SHA256SUMS`, formal-model logs, benchmark summaries, and deterministic vector outputs. Later repository revisions are not part of the evaluated snapshot.

Funding

The author declares that this research received no external funding.

CRediT Author Statement

Oleksandr Melnychenko: conceptualization, methodology, software, validation, formal analysis, investigation, writing–original draft, writing–review and editing, and visualization.

Declaration of Competing Interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of Generative AI and AI-assisted technologies in the writing process

OpenAI Codex was used during manuscript preparation to assist with English-language editing, consistency checks, and minor LaTeX typesetting support. OpenAI Codex was not used as an authoritative source for technical claims, references, equations, implementation results, or experimental measurements. All protocol descriptions, equations, references, tables, implementation details, and experimental results were independently reviewed, verified, and approved by the author, who takes full responsibility for the content of the article.

References

[1] M. Marlinspike and T. Perrin. The X3DH key agreement protocol, revision 1. Signal Technical Documentation, 2016. https://signal.org/docs/specifications/x3dh/

[2] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila. A formal security analysis of the Signal messaging protocol. *Journal of Cryptology*, 33(4):1914–1983, 2020. https://doi.org/10.1007/s00145-020-09360-1

[3] S. Svystun, L. Scisło, M. Pawlik, O. Melnychenko, P. Radiuk, O. Savenko, and A. Sachenko. DyTAM: Accelerating wind turbine inspections with dynamic UAV trajectory adaptation. *Energies*, 18(7):1823, 2025. https://doi.org/10.3390/en18071823

[4] S. Svystun, O. Melnychenko, P. Radiuk, O. Savenko, A. Sachenko, and A. Lysyi. Thermal and RGB images work better together in wind turbine damage detection. *International Journal of Computing*, 23(4):526–535, 2024. https://doi.org/10.47839/ijc.23.4.3752

[5] A. Lysyi, A. Sachenko, P. Radiuk, M. Lysyi, O. Melnychenko, O. Ishchuk, and O. Savenko. Enhanced fire hazard detection in solar power plants: an integrated UAV, AI, and SCADA-based approach. *Radioelectronic and Computer Systems*, (2):99–117, 2025. https://doi.org/10.32620/reks.2025.2.06

[6] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard (ML-KEM). FIPS 203, 2024. https://csrc.nist.gov/pubs/fips/203/final

[7] P. W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997. https://arxiv.org/abs/quant-ph/9508027

[8] E. Kret and R. Schmidt. The PQXDH key agreement protocol, revision 3. Signal Technical Documentation, 2024. https://signal.org/docs/specifications/pqxdh/

[9] G. Connell and R. Schmidt. Signal Protocol and post-quantum ratchets. Signal Blog, 2025. https://signal.org/blog/spqr/

[10] T. Perrin, M. Marlinspike, and R. Schmidt. The Double Ratchet Algorithm, revision 4. Signal Technical Documentation, 2025. https://signal.org/docs/specifications/doubleratchet/

[11] R. Schmidt. The ML-KEM Braid Protocol. Signal Technical Documentation, 2025. https://signal.org/docs/specifications/mlkembraid/

[12] Apple Security Engineering and Architecture. iMessage with PQ3: The new state of the art in quantum-secure messaging at scale. Apple Security Research, 2024. https://security.apple.com/blog/imessage-pq3/

[13] D. Stebila. Security analysis of the iMessage PQ3 protocol. Technical report, 2024. https://www.douglas.stebila.ca/research/papers/Apple-Stebila24/

[14] F. Linker, R. Sasse, and D. Basin. A Formal Analysis of Apple's iMessage PQ3 Protocol. In *USENIX Security 2025*, pages 5015–5034. https://www.usenix.org/conference/usenixsecurity25/presentation/linker

[15] J. Alwen, S. Coretti, and Y. Dodis. The double ratchet: Security notions, proofs, and modularization for the Signal protocol. In *EUROCRYPT 2019*, pages 129–158. Springer, 2019. https://doi.org/10.1007/978-3-030-17653-2_5

[16] A. Bienstock, J. Fairoze, S. Garg, P. Mukherjee, and S. Raghuraman. A more complete analysis of the Signal Double Ratchet algorithm. In *CRYPTO 2022*, pages 784–813. Springer, 2022. https://doi.org/10.1007/978-3-031-15802-5_27

[17] J. Brendel, M. Fischlin, F. Günther, C. Janson, and D. Stebila. Towards post-quantum security for Signal's X3DH handshake. In *Selected Areas in Cryptography – SAC 2020*, pages 404–430. Springer, 2021. https://doi.org/10.1007/978-3-030-81652-0_16

[18] K. Hashimoto, S. Katsumata, K. Kwiatkowski, and T. Prest. An efficient and generic construction for Signal's handshake (X3DH): Post-quantum, state leakage secure, and deniable. In *PKC 2021*, pages 410–440. Springer, 2021. https://doi.org/10.1007/978-3-030-75248-4_15

[19] A. Langley. CECPQ2. Google Chrome experiment, 2018. https://www.imperialviolet.org/2018/12/12/cecpq2.html

[20] D. Stebila and M. Mosca. Post-quantum key exchange for the Internet and the Open Quantum Safe project. In *Selected Areas in Cryptography – SAC 2016*, pages 14–37. Springer, 2017. https://doi.org/10.1007/978-3-319-69453-5_2

[21] N. Bindel, J. Brendel, M. Fischlin, B. Goncalves, and D. Stebila. Hybrid key encapsulation mechanisms and authenticated key exchange. In *Post-Quantum Cryptography – PQCrypto 2019*, pages 206–226. Springer, 2019. https://doi.org/10.1007/978-3-030-25510-7_12

[22] B. Dowling, M. Fischlin, F. Günther, and D. Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol. *Journal of Cryptology*, 34(4):37, 2021. https://doi.org/10.1007/s00145-021-09384-1

[23] S. Meier, B. Schmidt, C. Cremers, and D. Basin. The TAMARIN prover for the symbolic analysis of security protocols. In *CAV 2013*, pages 696–701. Springer, 2013. https://doi.org/10.1007/978-3-642-39799-8_48

[24] B. Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, 2016. https://bblanche.gitlabpages.inria.fr/publications/BlanchetFnTPS16.html

[25] J. Lund. Technology preview: Sealed Sender for Signal. Signal Blog, 2018. https://signal.org/blog/sealed-sender/

[26] G. Danezis and I. Goldberg. Sphinx: A compact and provably secure mix format. In *IEEE Symposium on Security and Privacy*, pages 269–282, 2009. https://doi.org/10.1109/SP.2009.15

[27] A. M. Piotrowska, J. Hayes, T. Elahi, S. Meiser, and G. Danezis. The Loopix anonymity system. In *USENIX Security 2017*, pages 1199–1216. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/piotrowska

[28] S. Gueron, A. Langley, and Y. Lindell. AES-GCM-SIV: Nonce misuse-resistant authenticated encryption. RFC 8452, IETF, 2019. https://www.rfc-editor.org/rfc/rfc8452

[29] P. Rogaway and T. Shrimpton. A provable-security treatment of the key-wrap problem. In *EUROCRYPT 2006*, pages 373–390. Springer, 2006. https://doi.org/10.1007/11761679_23

[30] A. Langley, M. Hamburg, and S. Turner. Elliptic curves for security. RFC 7748, IETF, 2016. https://www.rfc-editor.org/rfc/rfc7748

[31] H. Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. In *CRYPTO 2010*, pages 631–648. Springer, 2010. https://doi.org/10.1007/978-3-642-14623-7_34

[32] H. Krawczyk and P. Eronen. HMAC-based extract-and-expand key derivation function (HKDF). RFC 5869, IETF, 2010. https://www.rfc-editor.org/rfc/rfc5869

[33] B. LaMacchia, K. Lauter, and A. Mityagin. Stronger security of authenticated key exchange. In *ProvSec 2007*, pages 1–16. Springer, 2007. https://doi.org/10.1007/978-3-540-75670-5_1

[34] R. Canetti and H. Krawczyk. Analysis of key-exchange protocols and their use for building secure channels. In *EUROCRYPT 2001*, pages 453–474. Springer, 2001. https://doi.org/10.1007/3-540-44987-6_28

[35] National Institute of Standards and Technology. The keyed-hash message authentication code (HMAC). FIPS 198-1, 2008. https://doi.org/10.6028/NIST.FIPS.198-1

[36] S. Josefsson and I. Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, IETF, 2017. https://www.rfc-editor.org/rfc/rfc8032

[37] Signal Messenger. SparsePostQuantumRatchet. Public reference material, 2025. https://github.com/signalapp/SparsePostQuantumRatchet